EE-599: Designing and Building Autonomous AI Agents

University of Southern California

Ming Hsieh Department of Electrical and Computer Engineering

Week 8: Research Agents

Saving task progress and using memory across conversations

Instructor: Arash Saifhashemi

Research agents

Iterative search, evidence, and synthesis

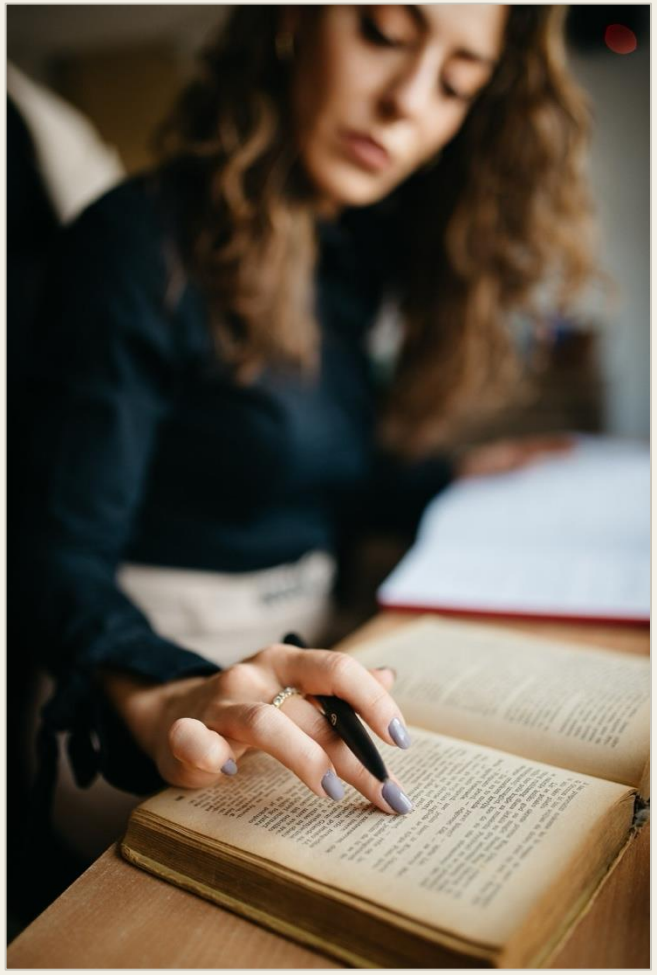
EE 599: Designing and Building Autonomous AI Agents

How does an agent decide what to investigate next?



What research agents do

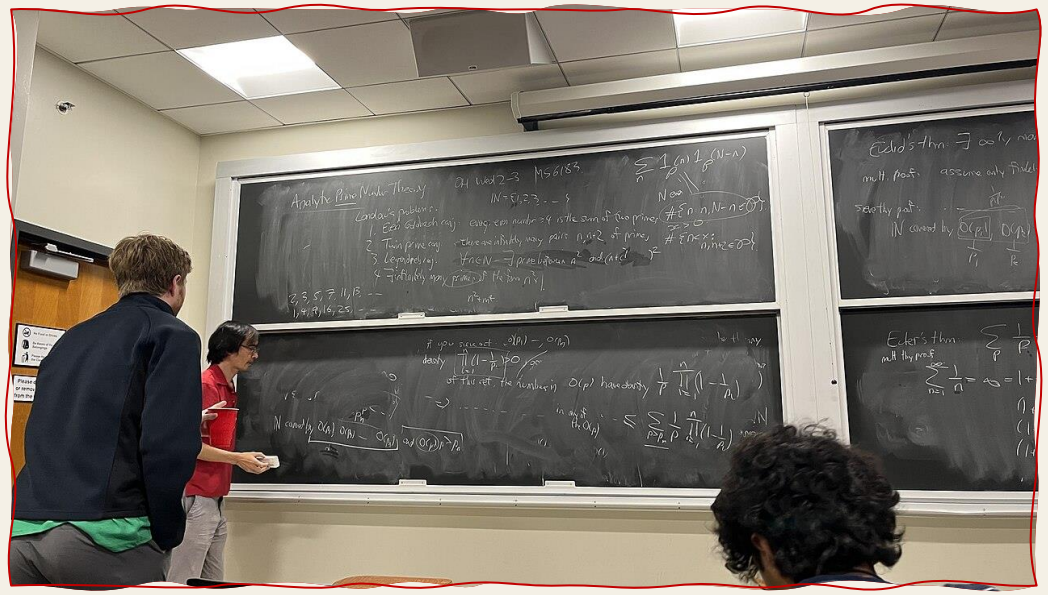
- A research agent investigates a question. It chooses **what to read** and **what to investigate next**.
- After each search, it reads the results and decides to:
  - search again
  - read a source more closely
  - check a claim
  - write the answer



The lecture path

1. **Using search results to choose the next step**
   RAG, ReAct, FLARE, Adaptive-RAG
2. **Researching before writing**
   STORM, Anthropic's multi-agent research system
3. **Investigating scientific literature**
   PaperQA2, citation checks
4. **Learning search behavior**
   Search-R1: training search with rewards
5. **Research on the open web**
   DeepResearcher
6. **Producing new research results**
   Aletheia, Self-Refine
7. **Choosing and testing a research agent**
   design choices, fair comparisons

Goal: judge what a research agent's results really show.



Running example: a battery-life question

A city bus company asks:

*"Does **fast charging** shorten the life of our **electric buses**' batteries, and under what conditions?"*

- The assistant must **find** studies that test this question.
- For each study, its report should give the **test conditions**, **results**, and **supporting passages**.



Our question as builders of the assistant is different: when does letting the assistant search again, instead of once, produce a better report?

01

Using search results to choose the next step

When should the assistant search again, read a source, or write its answer?

The bus company needs measured battery tests. Suppose the first search finds only charger buying guides. The assistant must search again.

Retrieval-augmented generation (RAG)

How RAG works

- It searches **once** and adds the returned passages to the prompt.
- It writes the answer from those passages and **never searches again**.

Where it fails: the bus company's question

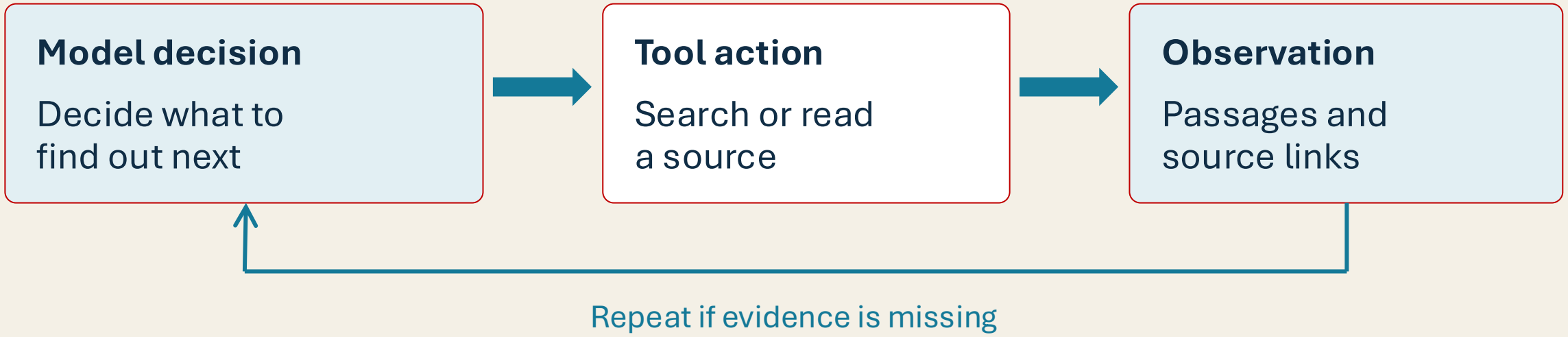
- The search "fast charging battery life" returns charger buying guides.
- The guides describe products but contain **no battery measurements**.
- RAG writes its report from them anyway.

Question: how could the assistant notice this and search again?

The research loop

An observation is **information returned by a tool**, such as retrieved passages.

The model reads this information before choosing its next action.



The search returned **buying guides**, not battery tests.
The assistant **searches again** for published measurements. It answers when supported or reports missing evidence.

ReAct: reasoning between tool actions

How ReAct works

- The model alternates **written reasoning** with **tool actions**.
- Each tool result informs the next reasoning step, so the model can change course.

Example: the bus company's question

1. **Action:** search *"fast charging battery life"*
2. **Observation:** the results are buying guides for chargers.
3. **Thought:** the report needs published tests of capacity loss at stated charging rates and temperatures. Search again.
4. **Action:** search *"lithium-ion battery capacity loss in fast-charging cycle tests at different temperatures"*

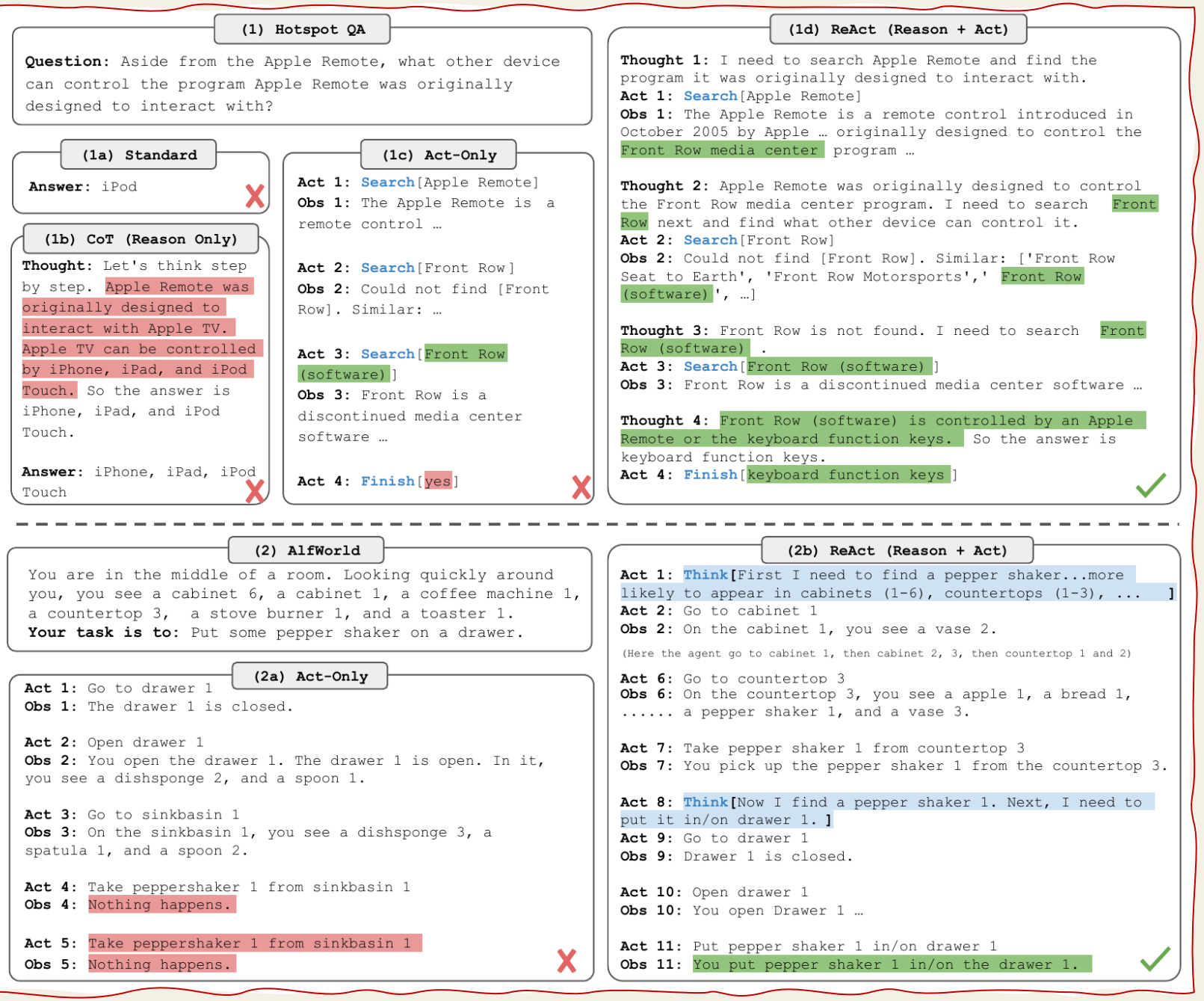
Reasoning and tool actions: ReAct §2.

Four ways to prompt the same model

PaLM-540B is the language model used in the paper's question-answering comparison. It has **540 billion** parameters. Its weights stay fixed while the prompt format changes.

1. **Standard:** give an answer without intermediate reasoning or tool calls.
2. **Chain-of-thought (CoT):** write intermediate reasoning steps, without calling tools in this baseline.
3. **Act-only:** use tool actions and observations, without explicit written reasoning between them.
4. **ReAct:** interleave written reasoning, tool actions, and observations.

Prompting baselines and model: Yao et al. (2023), §3.2 and Table 1.



Compare each method on the left with **ReAct** on the right.

Top: HotpotQA

A multi-step question that needs Wikipedia facts.

- **CoT** assumes the program is Apple TV and answers “iPhone, iPad, and iPod Touch.”
- **ReAct** searches, finds Front Row, and answers “keyboard function keys.”

Bottom: ALFWorld

A room described in text.

- **Act-only** repeats a failed action.
- **ReAct** finds the pepper shaker and puts it on a drawer.

Figure 1: prompting and action examples, Yao et al. (2023).

What do the ReAct scores measure?

HotpotQA: answer a question using multiple passages

- **Metric: exact match (EM).** The percentage of questions where the model's answer is the same as the correct answer, ignoring capital letters and punctuation.

FEVER: check a factual claim against Wikipedia

- **Metric: accuracy (Acc).** The percentage of claims given the correct verdict.
- This tests classifying claims, not matching an answer phrase.

Reading the table

- Both scores are percentages; higher is better.
- The tasks differ, so compare methods within each column.

Tasks and metrics for the prompting comparison: Yao et al. (2023), §3 and Table 1.

CoT-SC: comparing reasoning attempts

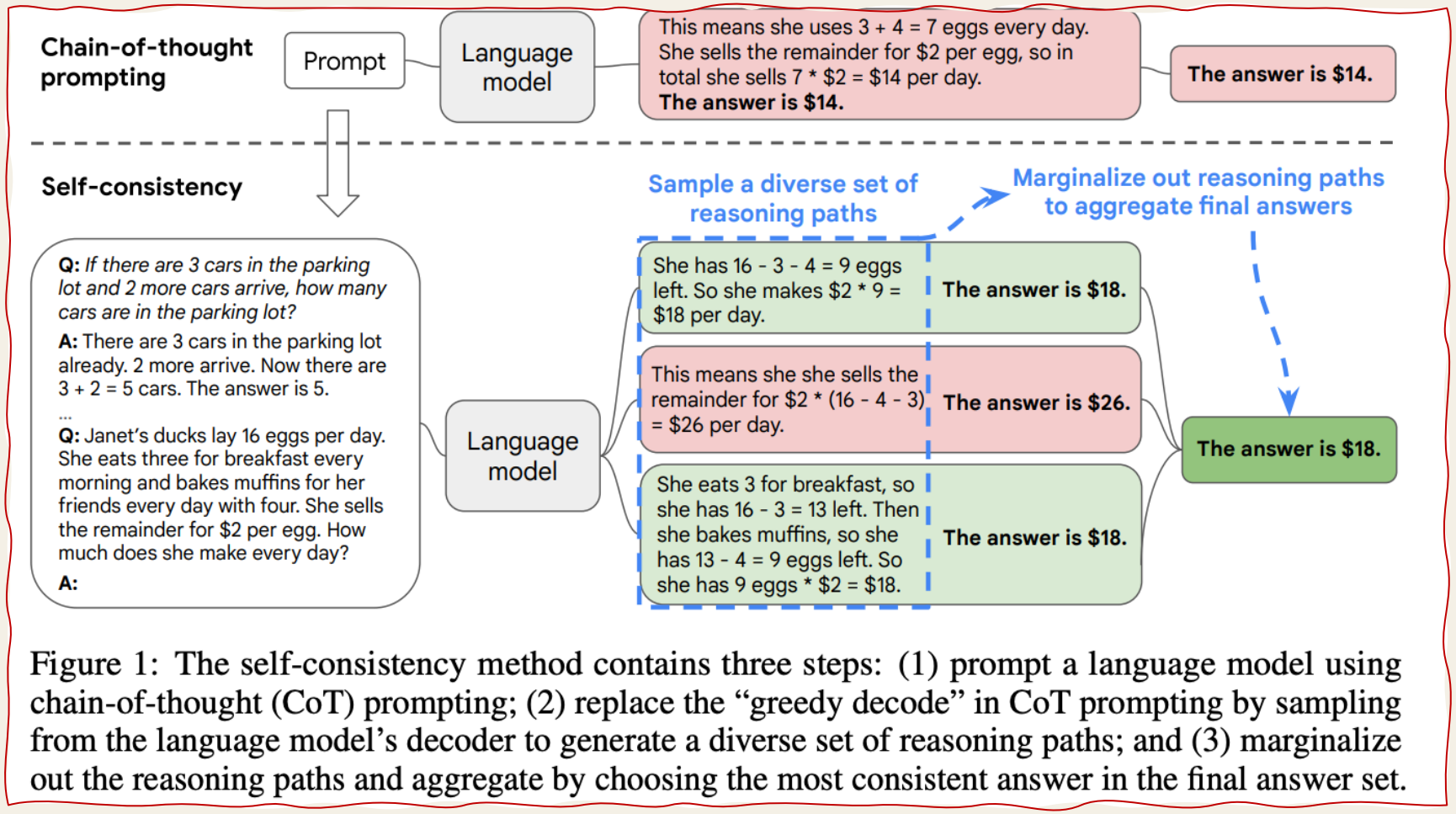


Figure 1: The self-consistency method contains three steps: (1) prompt a language model using chain-of-thought (CoT) prompting; (2) replace the "greedy decode" in CoT prompting by sampling from the language model's decoder to generate a diverse set of reasoning paths; and (3) marginalize out the reasoning paths and aggregate by choosing the most consistent answer in the final answer set.

Self-consistency baseline and 21-sample setting: Yao et al. (2023), §3.2.

Combining ReAct and self-consistency

Idea: each method fails differently, so run one first and switch to the other when a simple signal shows it is struggling.

ReAct first, CoT-SC as backup

- **Switch when** ReAct has no answer after 7 steps (HotpotQA) or 5 steps (FEVER).
- **Then** sample 21 reasoning attempts and take the most frequent answer.

CoT-SC first, ReAct as backup

- **Switch when** the top answer gets fewer than half of the 21 votes.
- **Then** run ReAct to search for the missing facts.

Example: the Apple Remote question, CoT-SC first

- Of 21 attempts, 8 say “iPhone, iPad, iPod Touch,” 7 say “iPod,” and 6 say something else.
- No answer has a majority, so the system switches to ReAct, which searches and finds Front Row.

Fallback rules: Yao et al. (2023), §3.2.

ReAct: task-dependent gains

Prompt Method[a]	HotpotQA (EM)	Fever (Acc)
Standard	28.7	57.1
CoT (Wei et al., 2022)	29.4	56.3
CoT-SC (Wang et al., 2022a)	33.4	60.4
Act	25.7	58.9
ReAct	27.4	60.9
CoT-SC → ReAct	34.2	**64.6**
ReAct → CoT-SC	**35.1**	62.0
Supervised SoTA[b]	67.5	89.5

Table 1: PaLM-540B prompting results on HotpotQA and Fever.

[a] HotpotQA EM is 27.1, 28.9, 33.8 for Standard, CoT, CoT-SC in Wang et al. (2022b).

[b] (Zhu et al., 2021; Lewis et al., 2020)

ReAct vs. CoT

- HotpotQA: 27.4% vs. **29.4%**, so CoT is higher.
- FEVER: **60.9%** vs. 56.3%, so ReAct is higher.

What the table shows

- Neither method wins on both tasks.
- The hybrid methods score highest among the prompting methods.

Takeaway: reasoning with tools can help, but the gain depends on the task.

Supervised SoTA is the best published score from systems trained specifically for each task.

Table 1: PaLM-540B prompting results, Yao et al. (2023).

How ReAct and chain-of-thought fail

Searching changes *how* a model fails, not *whether* it fails.

	Type	Definition	ReAct	CoT
Success	True positive	Correct reasoning trace and facts	94%	86%
	False positive	Hallucinated reasoning trace or facts	6%	14%
Failure	Reasoning error	Wrong reasoning trace (including failing to recover from repetitive steps)	47%	16%
	Search result error	Search return empty or does not contain useful information	23%	-
	Hallucination	Hallucinated reasoning trace or facts	0%	56%
	Label ambiguity	Right prediction but did not match the label precisely	29%	28%

Table 2: Types of success and failure modes of ReAct and CoT on HotpotQA, as well as their percentages in randomly selected examples studied by human.

Failure analysis: Yao et al. (2023), §3.3 and Table 2.

How ReAct and chain-of-thought fail

Searching changes *how* a model fails, not *whether* it fails.

	Type	Definition	ReAct	CoT
Success	True positive	Correct reasoning trace and facts	94%	86%
	False positive	Hallucinated reasoning trace or facts	6%	14%
Failure	Reasoning error	Wrong reasoning trace (including failing to recover from repetitive steps)	47%	16%
	Search result error	Search return empty or does not contain useful information	23%	-
	Hallucination	Hallucinated reasoning trace or facts	0%	56%
	Label ambiguity	Right prediction but did not match the label precisely	29%	28%

Table 2: Types of success and failure modes of ReAct and CoT on HotpotQA, as well as their percentages in randomly selected examples studied by human.

- **Chain-of-thought:** when it's wrong, it's usually because it made up a fact.
- **ReAct:** rarely makes things up, because it works from what it finds. It gets stuck when a search finds nothing useful or it repeats the same step.
- **Hybrids:** switch to the other approach when one is struggling, so they score highest.

How ReAct and chain-of-thought fail

Searching changes *how* a model fails, not *whether* it fails.

	Type	Definition	ReAct	CoT
Success	True positive	Correct reasoning trace and facts	94%	86%
	False positive	Hallucinated reasoning trace or facts	6%	14%
Failure	Reasoning error	Wrong reasoning trace (including failing to recover from repetitive steps)	47%	16%
	Search result error	Search return empty or does not contain useful information	23%	-
	Hallucination	Hallucinated reasoning trace or facts	0%	56%
	Label ambiguity	Right prediction but did not match the label precisely	29%	28%

Table 2: Types of success and failure modes of ReAct and CoT on HotpotQA, as well as their percentages in randomly selected examples studied by human.

- **Chain-of-thought:** when it's wrong, it's usually because it made up a fact.
- **ReAct:** rarely makes things up, because it works from what it finds. It gets stuck when a search finds nothing useful or it repeats the same step.
- **Hybrids:** switch to the other approach when one is struggling, so they score highest.

With search, check both the evidence and the search process.

Adaptive retrieval: deciding when to search

RAG searches once, before it starts writing.

Adaptive retrieval decides while writing whether to search again

Jiang et al. (2023), FLARE; Asai et al. (2024), Self-RAG; Jeong et al. (2024), Adaptive-RAG.

FLARE

Drafts the next sentence. If the draft contains low-confidence tokens, it uses the draft as a search query, retrieves, and rewrites the sentence.

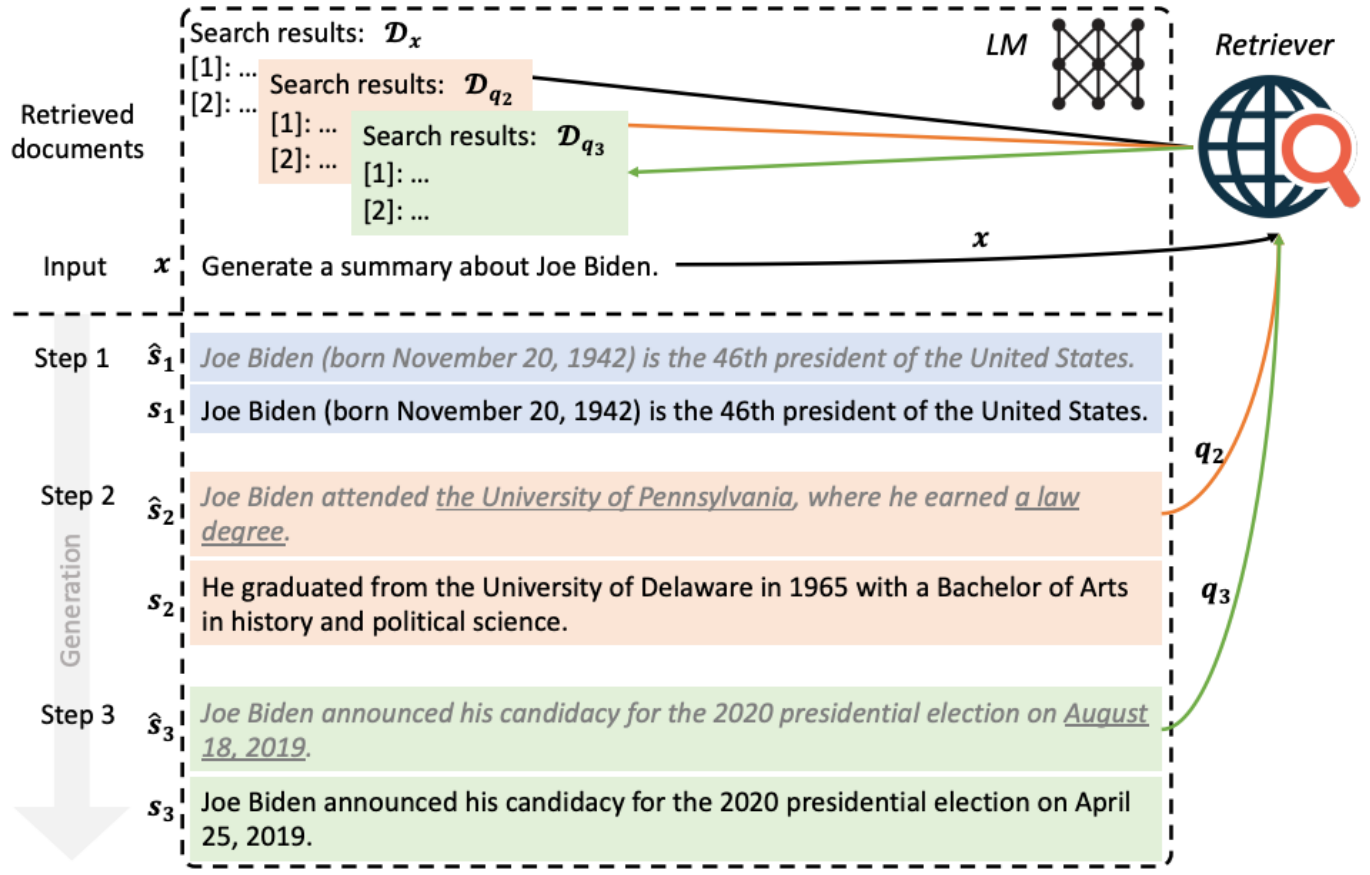


Figure 1: An illustration of forward-looking active retrieval augmented generation (FLARE). Starting with the user input x and initial retrieval results $\mathcal{D}_x$, FLARE iteratively generates a temporary next sentence (shown in *gray italic*) and check whether it contains low-probability tokens (indicated with underline). If so (step 2 and 3), the system retrieves relevant documents and regenerates the sentence.

Jiang et al. (2023), FLARE.

FLARE-Instruct

Note: The model is prompted to return search queries.

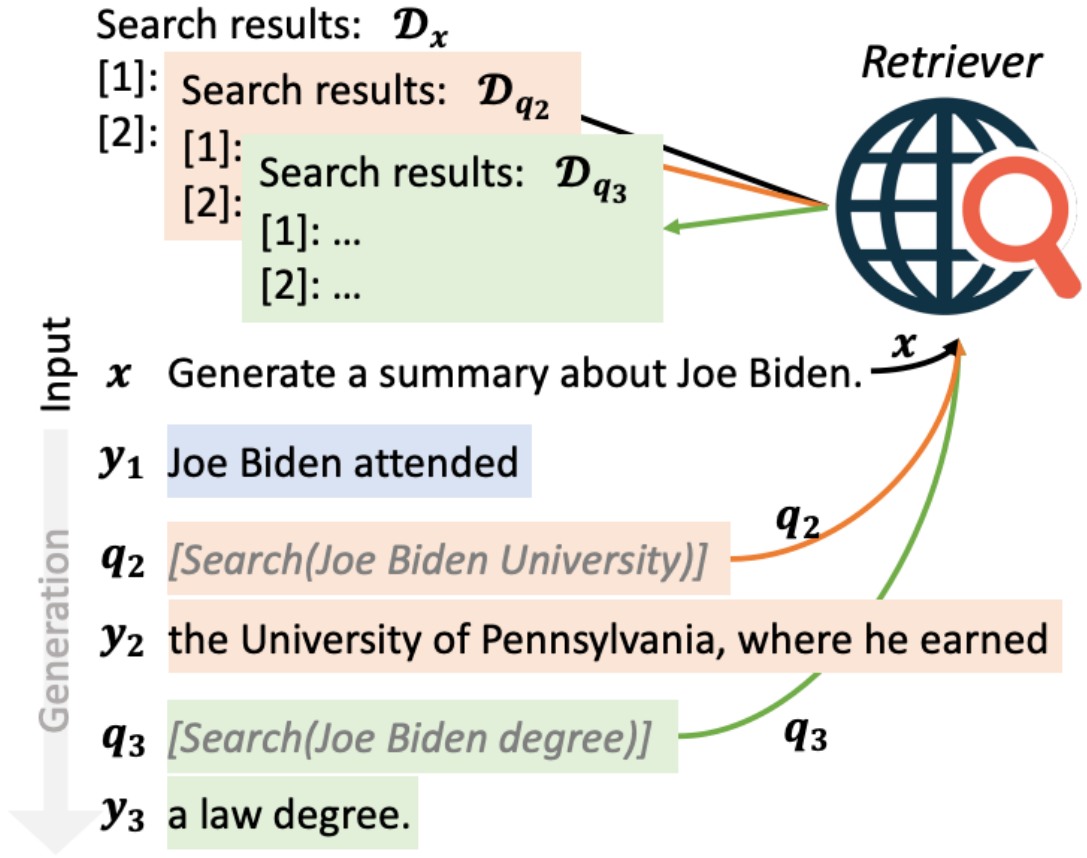


Figure 2: An illustration of forward-looking active retrieval augmented generation with retrieval instructions ($\text{FLARE}_{\text{instruct}}$). It iteratively generates search queries (shown in *gray italic*) to retrieve relevant information to aid future generations.

Jiang et al. (2023), FLARE (FLARE-Instruct).

Adaptive RAG

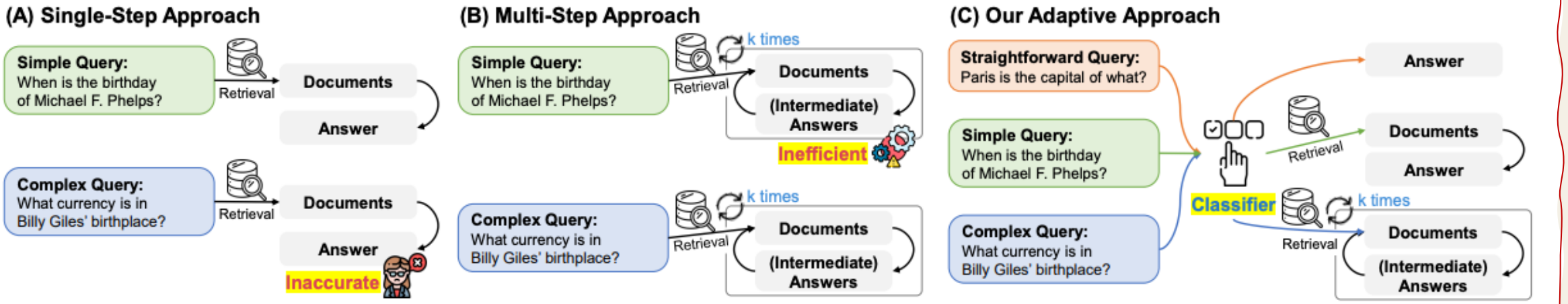


Figure 2: A conceptual comparison of different retrieval-augmented LLM approaches to question answering. (A) In response to a query, this single-step approach retrieves relevant documents and then generates an answer. However, it may not be sufficient for complex queries that require multi-step reasoning. (B) This multi-step approach iteratively retrieves documents and generates intermediate answers, which is powerful yet largely inefficient for the simple query since it requires multiple accesses to both LLMs and retrievers. (C) Our adaptive approach can select the most suitable strategy for retrieval-augmented LLMs, ranging from iterative, to single, to even no retrieval approaches, based on the complexity of given queries determined by our classifier.

A small classifier predicts how complex a question is and routes it to no retrieval, a single search, or repeated searches.

Jeong et al. (2024), Adaptive-RAG.

When should the agent stop searching?

Stop searching when one of three rules applies:

1. **Enough evidence:** every part of the question has a supporting source.
   - For the bus company: charging rate, temperature, and cost.
2. **No new information:** new searches keep returning sources the agent has already read.
   - More searching is unlikely to change the answer.
3. **Budget reached:** a set number of tool calls, amount of time, or tokens is used up.
   - ReAct's fallback rule gave up after 7 steps on HotpotQA and 5 on FEVER.

Whichever rule stops the run, the report should say what is still missing.

02

Researching before writing

How do several investigations become one coherent report?

The assistant has found battery studies. It needs an outline that shows when fast charging causes harm.

Research perspectives and questions

Perspective: a way of looking at the topic.

- Different readers need **different evidence**, so one broad search can leave important questions out.

For the bus company's report

- **Battery researcher:** which charging rates and temperatures cause the most wear?
- **Fleet manager:** how often will batteries need replacing, and at what cost?
- **Safety engineer:** does fast charging raise the risk of overheating?



Then: each question guides a separate investigation before the report is drafted.

From readers' questions to a report outline

Question: does fast charging wear out bus batteries?

The assistant turns each reader's question into a section of the report:

Section	Who asked	Evidence so far
1. Effects by temperature	Battery researcher	Two lab studies
2. Effects by charging rate	Battery researcher	Three lab studies
3. Replacement cost	Fleet manager	Nothing found yet
4. Overheating risk	Safety engineer	One safety report

- The summary is written last, from these four sections.
- **Gaps stay visible:** nothing on replacement cost yet, so the report says "not reported" instead of guessing.

Writing one section: compare studies under the same conditions

Section 1 of the outline (effects by temperature) cites two studies that seem to disagree:

Study	Cell type	Charge rate	Temperature	Cycles	Capacity loss	Cost
A	NMC	3C	**0 °C**	500	**18%**	not reported
B	NMC	3C	**25 °C**	500	**6%**	not reported

NMC: a common lithium-ion cell type. 3C: a charge rate that would fill the battery in about 20 minutes.

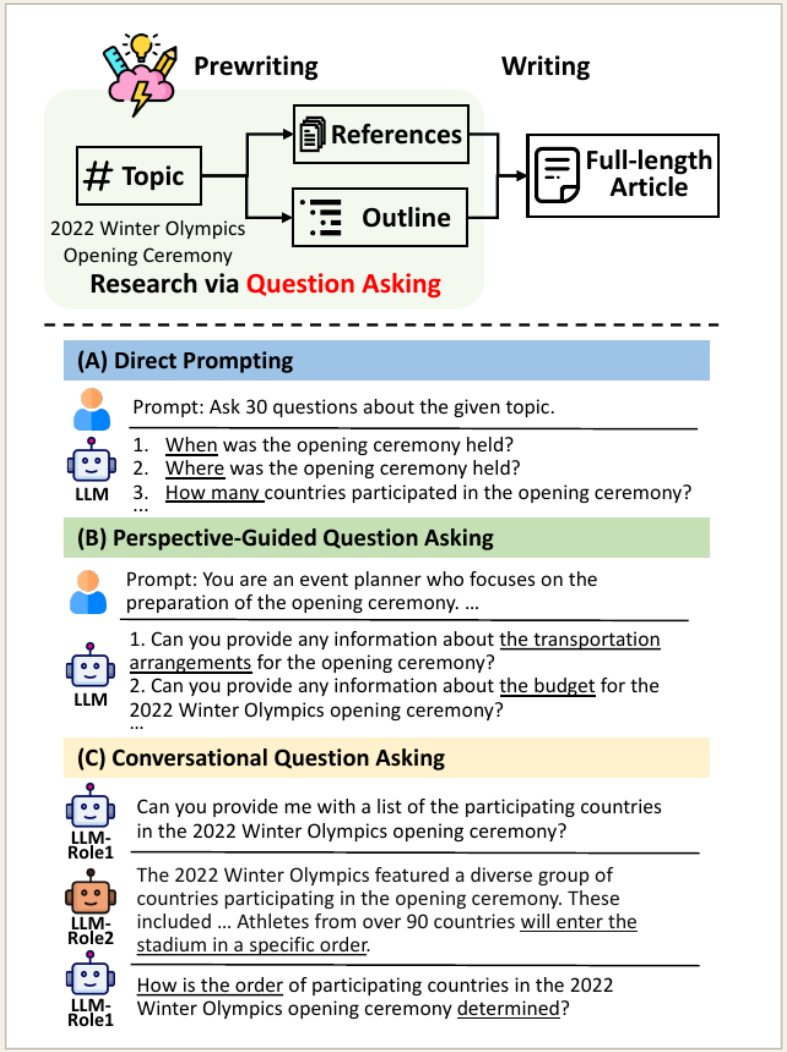
Why they differ

- Study A was tested at **0 °C**, study B at **25 °C**.
- Charge rate and cycles were the same, so temperature is the likely cause of the gap.

Two habits that make the comparison possible

- **Record the conditions** of every study, not just its result.
- **Cite the passage** for every number.

STORM: why question choice matters



Topic: the 2022 Winter Olympics opening ceremony

Three ways to generate questions

- **A. Direct prompting:** basic questions such as when, where, and how many countries.
- **B. A perspective:** an event planner adds questions about **transportation and budget**.
- **C. A conversation:** an answer mentions the countries' entry order, which prompts a follow-up: what determines the order?

Takeaway: perspectives and follow-ups find questions that direct prompting misses.

Figure 1: perspective and conversation examples, Shao et al. (2024).

How STORM finds better questions

Figure 1 compares three ways to ask about the Olympics opening ceremony:

Approach in Figure 1	What the writer asks	In STORM?
A. Ask directly	Basic questions: when, where, who took part	No: baseline
B. Add a perspective	An event planner asks about transport and budget	Yes: step 1
C. Follow up on answers	The answer mentions entry order → why that order?	Yes: step 2

STORM uses B and C together. A is the baseline it improves on.

Both roles are models: the “expert” searches sources; it is not a human.

Question examples: Shao et al. (2024), Figure 1 and §3.1–3.2.

STORM: research before drafting

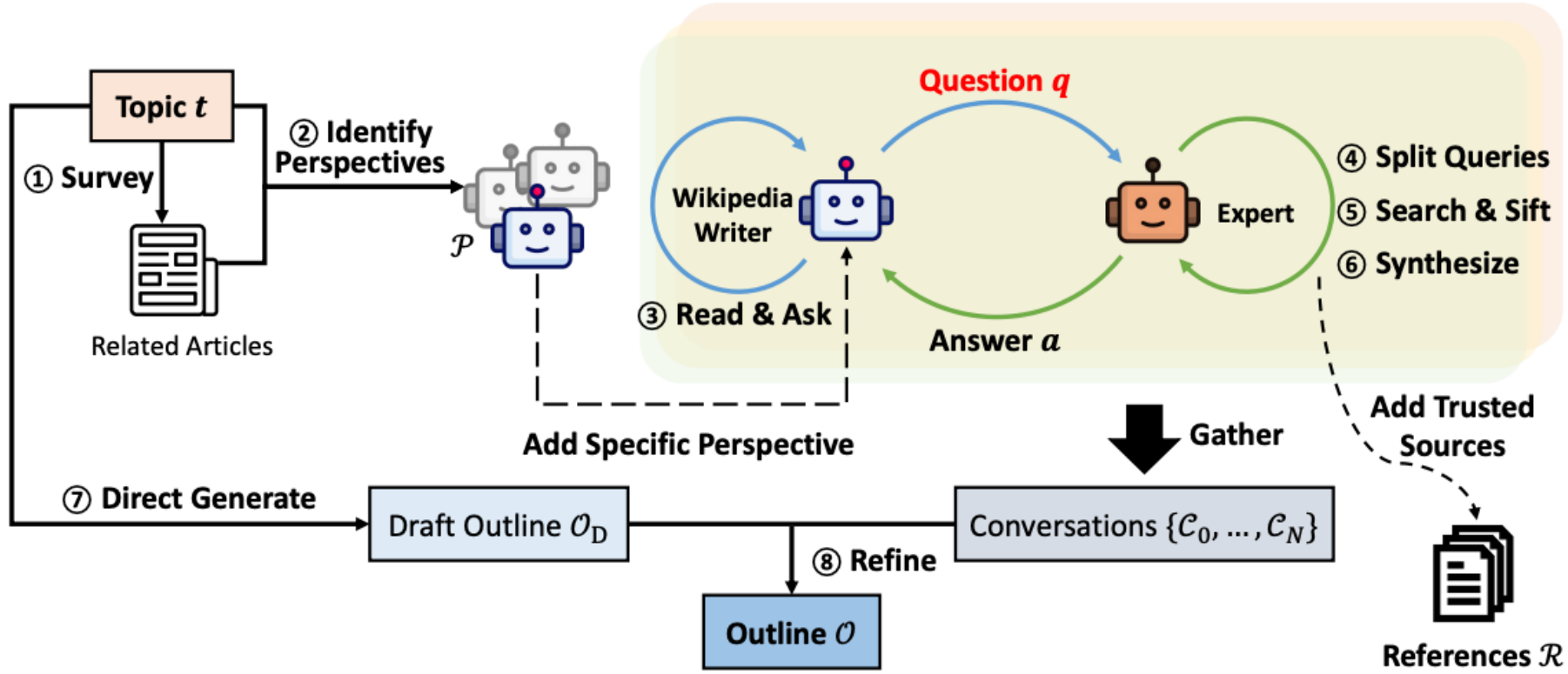


Figure 2: The overview of STORM that automates the pre-writing stage. Starting with a given topic, STORM identifies various perspectives on covering the topic by surveying related Wikipedia articles (①-②). It then simulates conversations between a Wikipedia writer who asks questions guided by the given perspective and an expert grounded on trustworthy online sources (③-⑥). The final outline is curated based on the LLM's intrinsic knowledge and the gathered conversations from different perspectives (⑦-⑧).

Evaluating the outline before the article

Goal: do STORM's outline headings cover the topic's important aspects as well as the headings of human-written Wikipedia articles?

Two measures

- **Heading soft recall:** compares heading meanings using **sentence embeddings**, which represent text as numerical vectors.
- **Heading entity recall:** the percentage of **human-heading names** that the outline also contains.

Reading the next table

- "w/o" means **without**: the authors remove perspectives or conversations, keeping the number of questions the same.
- The GPT-3.5 and GPT-4 blocks change the model used to draft and refine the outline.

Outline measures and controlled variants: Shao et al. (2024), §2.2, §4.4 and §5.2.

STORM: outline coverage results

		Heading Soft Recall	Heading Entity Recall
GPT-3.5	Direct Gen	80.23	32.39
	RAG/oRAG	73.59	33.85
	RAG-expand	74.40	33.85
	STORM	**86.26†**	**40.52†**
	w/o Perspective	84.49	40.12
	w/o Conversation	77.97	31.98
GPT-4	Direct Gen	87.66	34.78
	RAG/oRAG	89.55	42.38
	RAG-expand	91.36	43.53
	STORM	**92.73†**	**45.91**
	w/o Perspective	92.39	42.70
	w/o Conversation	88.75	39.30

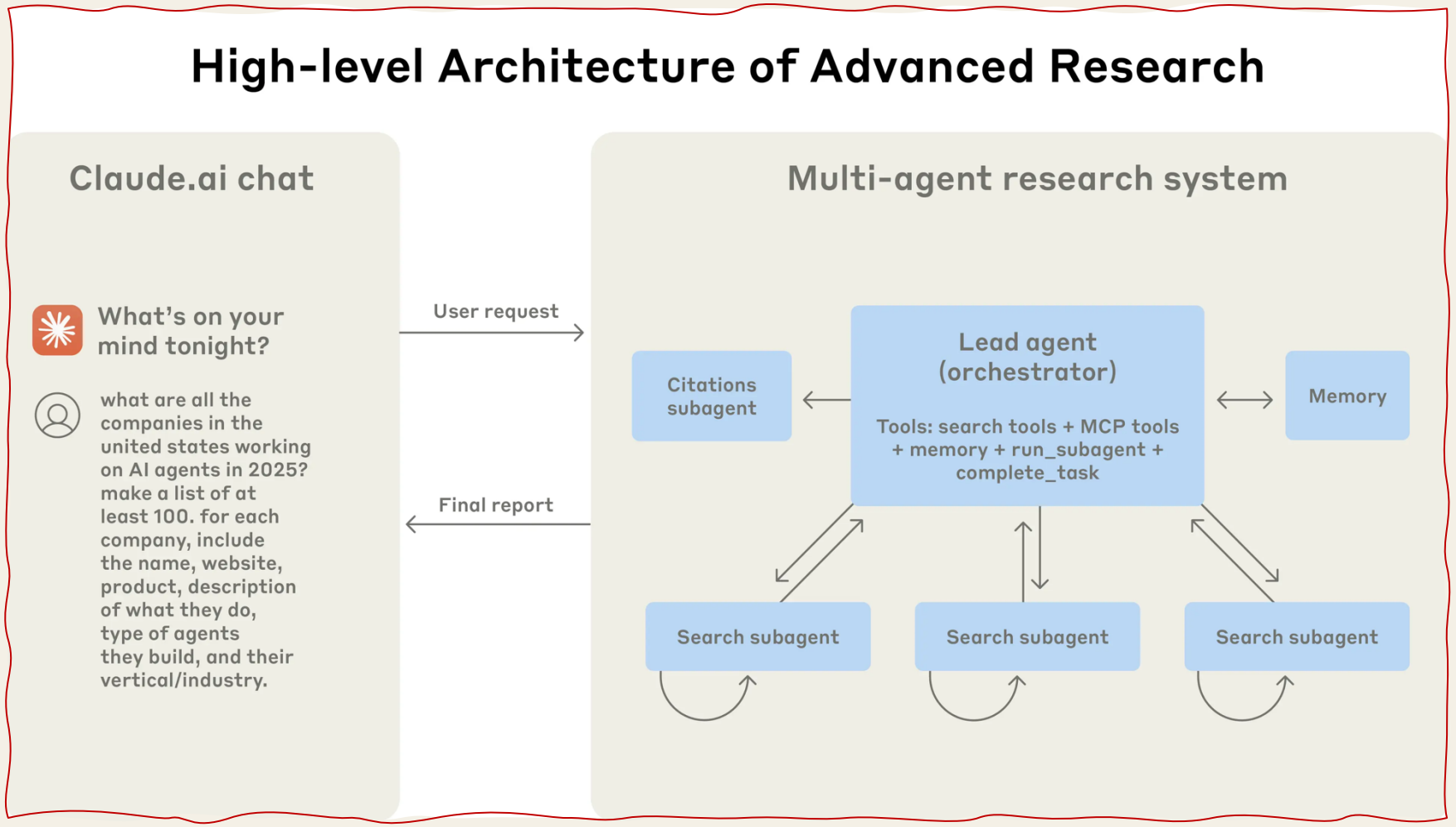
Table 3: Results of outline quality evaluation (%). † denotes significant differences ($p < 0.05$) from a paired t-test between STORM and baselines.

Baselines in the table

- **Direct Gen:** writes the outline from the topic alone.
- **RAG/oRAG:** searches once, then writes the outline (oRAG is outline-driven RAG).
- **RAG-expand:** searches again using the outline headings.

What the table shows

- STORM scores highest in every column.
- Removing conversations lowers STORM's scores more than removing perspectives.
- † marks a significant gain over the baselines ($p < 0.05$). With GPT-4, the entity-recall gain is not marked.



- A lead agent splits the question and runs search subagents in parallel, each with its own context.
- A citations subagent then links the report's claims to sources.

Hadfield et al. (2025), "How we built our multi-agent research system," Anthropic Engineering.

What parallel research costs

Anthropic reports these numbers from its own internal tests, not from a peer-reviewed study.

A lead agent with subagents, compared with alternatives:

Score on Anthropic's internal research test

+90.2%

Opus 4 lead with Sonnet 4 subagents, vs. one Opus 4 agent working alone

Tokens used

15×

more than a chat (a single agent: 4×)

The gain comes mainly from separate contexts and more tokens

Why: on BrowseComp, a benchmark of hard-to-find web facts, token usage alone explained 80% of the variation in performance.

Good fit: open-ended research that splits into independent parts.
Poor fit: tasks that need shared context, such as most coding.

Hadfield et al. (2025), Anthropic Engineering, June 13, 2025.

03

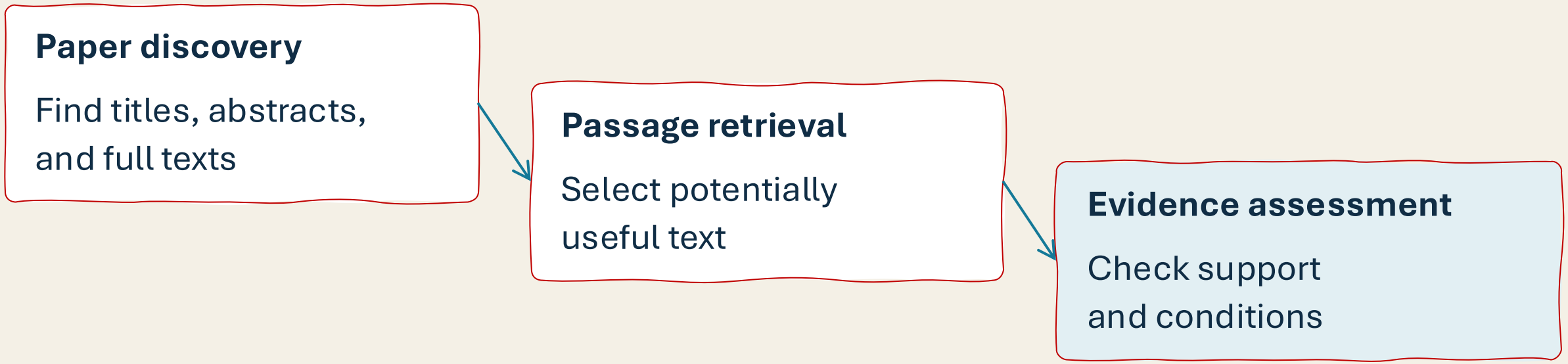
Investigating scientific literature

Which passage supports the precise claim we want to make?

The outline is ready. Each claim now needs a supporting passage. A relevant paper is not yet proof.

Finding papers and finding evidence

- The assistant first finds **candidate papers**, then selects passages relevant to the question.
- It must still check whether a passage **supports the precise claim** it wants to write.



Why the last step matters

- A paper's title can match the topic while its results fail to support a claim.
- Keeping the exact passage and its location lets a reader check that support.

Relevance and claim support

> Draft claim: "*STORM improves both article **organization** and **verifiability**.*"

- **Organization:** oRAG 3.25, STORM 4.00 (p = .005).
  If the systems were equal, a gap this large would occur only 0.5% of the time: likely a real gain.
- **Verifiability:** oRAG 3.85, STORM 3.80 (p = .843).
  A gap this small is common by chance: no evidence of a difference.

Revision: STORM improved editor-rated organization in this comparison.
It did not show better verifiability.

The paper is relevant to both parts of the claim, but these results support only the organization improvement, so the claim must be narrowed.

Claim check using Table 6.

Measuring whether citations support claims

ALCE (Gao et al., 2023) is a benchmark for answers with citations.

- It scores citations with an **NLI model**: a classifier that judges whether one text entails, that is, logically supports, another.

Two scores

- **Citation recall:** does each statement cite passages that together support it? Each statement scores 1 or 0; the score is the average over statements.
- **Citation precision:** is each citation needed? A citation that supports nothing on its own, and whose removal loses no support, counts as irrelevant.

Finding: even the best models lacked complete citation support 50% of the time on ELI5, a dataset of long, open-ended answers.

Gao et al. (2023), Enabling LLMs to Generate Text with Citations, EMNLP 2023.

Answering when evidence is missing

Abstention: declining to give an answer that the available evidence does not support.

- The assistant explains the gap and keeps investigating.

Example: the fleet manager's question

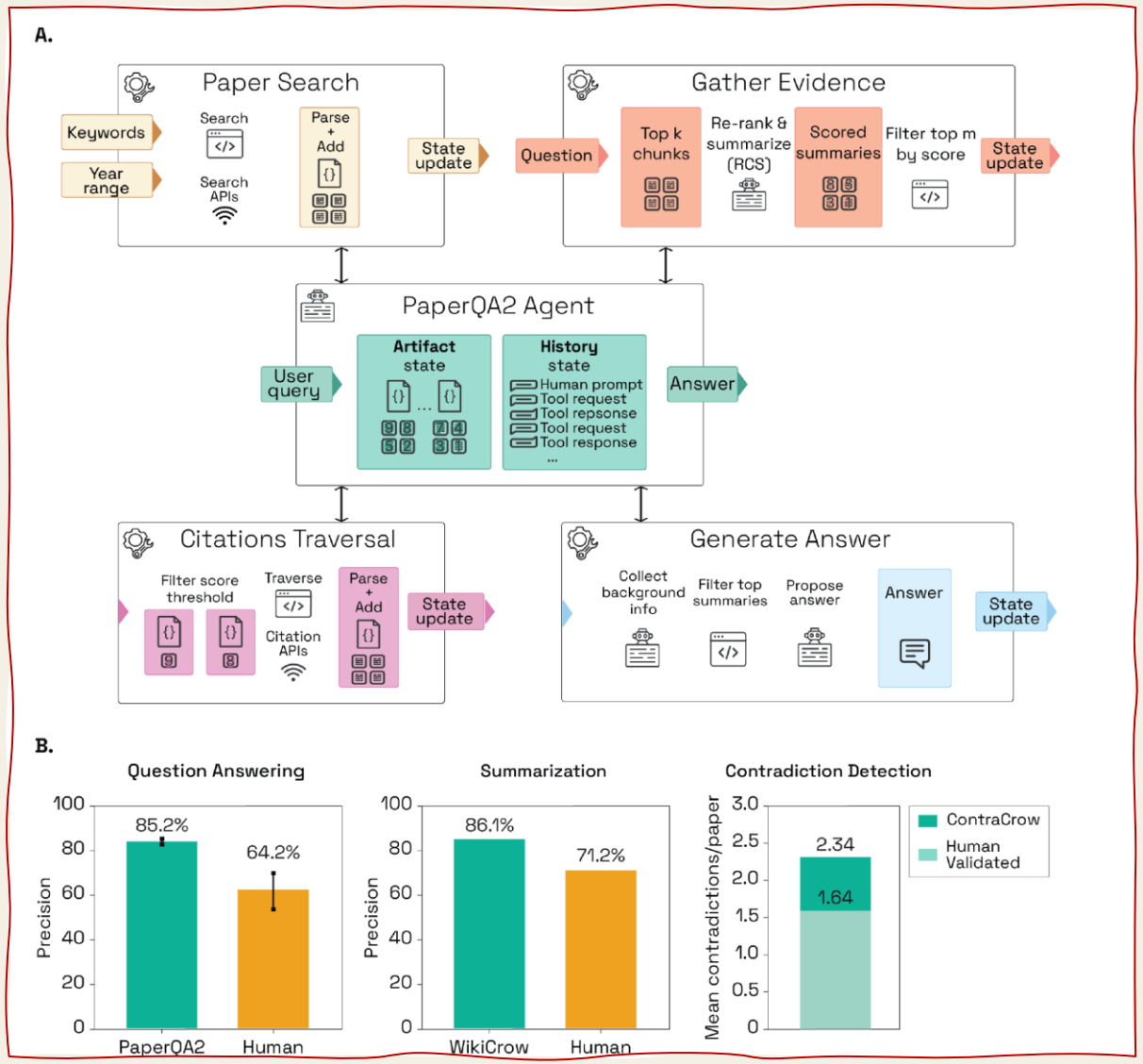
"Will fast charging raise our battery costs?"

- The assistant has found only **lab tests of capacity loss**.
- They report no **prices, bus schedules, or warranties**.

A useful response names what is missing

"These lab tests show capacity loss, not cost, so I cannot estimate cost yet. I need replacement prices and your charging schedule before recommending."

PaperQA2: investigating literature



One model chooses four tools

- **Paper Search:** finds papers and extracts their text.
- **Gather Evidence:** ranks and summarizes the most relevant passages.
- **Citations Traversal:** follows a paper's references and the later papers that cite it.
- **Generate Answer:** writes from the gathered evidence. If it is not enough, the model can search again.

Figure 1; Skarlinski et al. (2024).

An answer should be no broader than its evidence

PaperQA2's Figure 2A traces one question to its answer and the evidence behind it.

Step	In the paper's example
Question	Does a treatment change the amount of CD8α, a protein on certain T cells?
PaperQA2's answer	"No **significant** impact," citing Kellner et al. (2024)
The evidence	One study measured CD8α levels and found no significant change

Supported: no change in this one measurement, in this experiment.
Not supported: the treatment has no biological effect at all.

PaperQA2's answer stays within its evidence, i.e., it does not say no evidence at all.

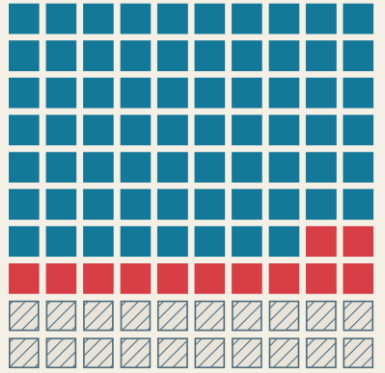
Figure 2A: question and evidence, Skarlinski et al. (2024).

Precision and accuracy: a worked example

In a 100-question test, the assistant answers 80 questions and gets 68 right. It leaves the other 20 questions unanswered.

Each square is one question.

68 correct answers
12 incorrect answers
20 unanswered (hatched)



Precision

Of the 80 answered questions, 68 are correct.

Precision = 68 / **80** = 85%.

Accuracy

Of all 100 questions, 68 are correct.

Accuracy = 68 / **100** = 68%.

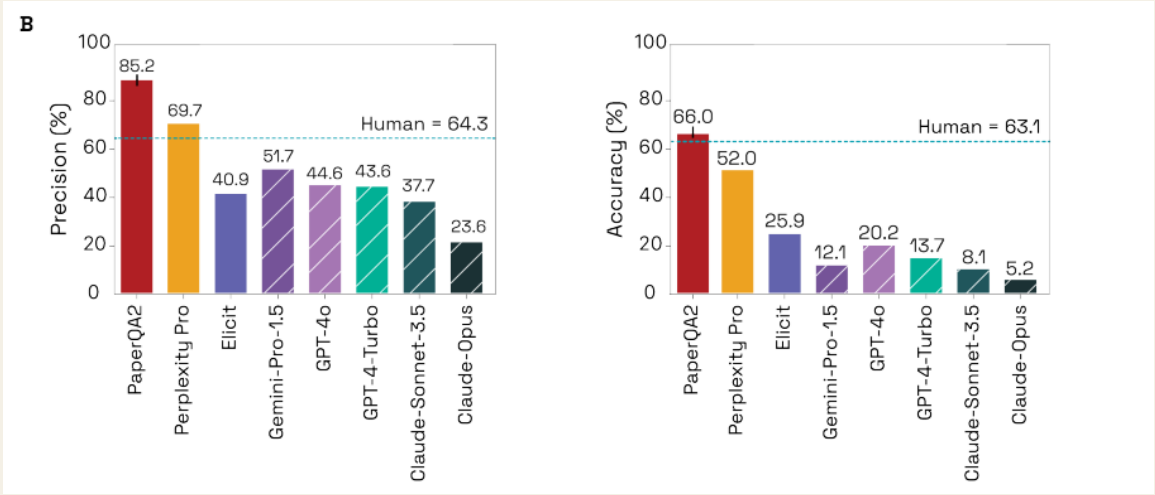
An assistant can have high precision while leaving many questions unanswered.

We need **both measures** to understand what the system delivers.

PaperQA2: precision and accuracy

LitQA2 contains 248 multiple-choice questions that require evidence from paper bodies.
Precision counts correct answers among the questions the system answered.
Accuracy includes every question, so unanswered questions also affect the score.

Reported outcome	PaperQA2	Human participants
Precision	85.2%	64.3%
Accuracy	66.0%	63.1%



PaperQA2's precision advantage **was significant** ($p = .0029$). Its accuracy advantage **was not** ($p = .63$). *Answering fewer questions* can improve precision without improving the number of correct answers across the whole test.

Reported means and comparison tests: Skarlinski et al. (2024), §2.

04

Learning search behavior and using tools

Can experience train the choice of the next search?

So far, a prompt tells the model to revise its searches.

Can feedback instead train the choices the model makes?

The Training Template

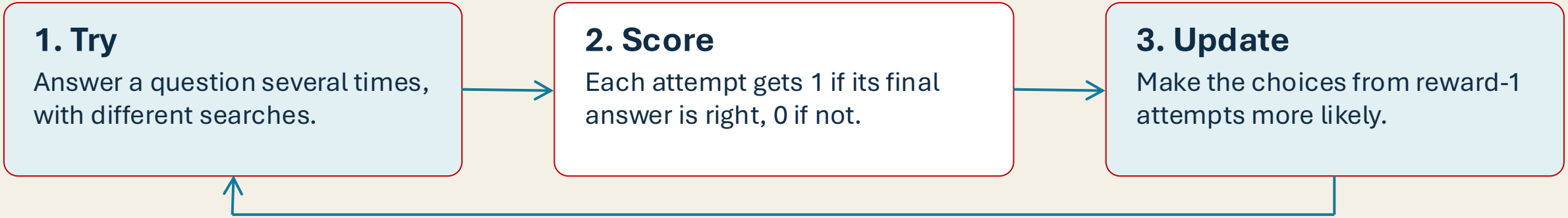
Answer the given question. You must conduct reasoning inside <think> and </think> first every time you get new information. After reasoning, if you find you lack some knowledge, you can call a search engine by <search> query </search>, and it will return the top searched results between <information> and </information>. You can search as many times as you want. If you find no further external knowledge needed, you can directly provide the answer inside <answer> and </answer> without detailed illustrations. For example, <answer> Beijing </answer>. Question: question.

Table 1: Template for SEARCH-R1. question will be replaced with the specific question during training and inference.

Reward-based policy training: Search-R1 §3, Jin et al. (2025).

Teaching the model to use tools with reinforcement learning

Training teaches the model **which searches lead to correct answers**. Each round has three steps:



Key point: nothing rewards searching itself. A search pays off only if it leads to a correct answer.

Example: Attempt B (next slide) found STORM Table 6, answered correctly, and earned reward 1.

In RL terms, the model's rule for choosing its next step is its policy, and the score is its reward.

Reward-based policy training: Search-R1 §3, Jin et al. (2025).

Scoring an attempt by its final answer

Example

- **Question:** which system received higher organization ratings in STORM's study?
- **Reward rule:** the reference answer is "STORM". A matching answer earns **1**; anything else earns **0**.

Two attempts

- **Attempt A** searches *"research agent leaderboard"* and answers "oRAG". Wrong: **reward 0**.
- **Attempt B** searches *"STORM Table 6 organization"* and answers "STORM". Correct: **reward 1**.

Limits of this reward

- It scores only the final answer, so it cannot tell which search step helped.
- It never checks whether the answer's citations support its claims.

Reward rule adapted from Search-R1 §3.4; answer comes from STORM Table 6.

Case Study

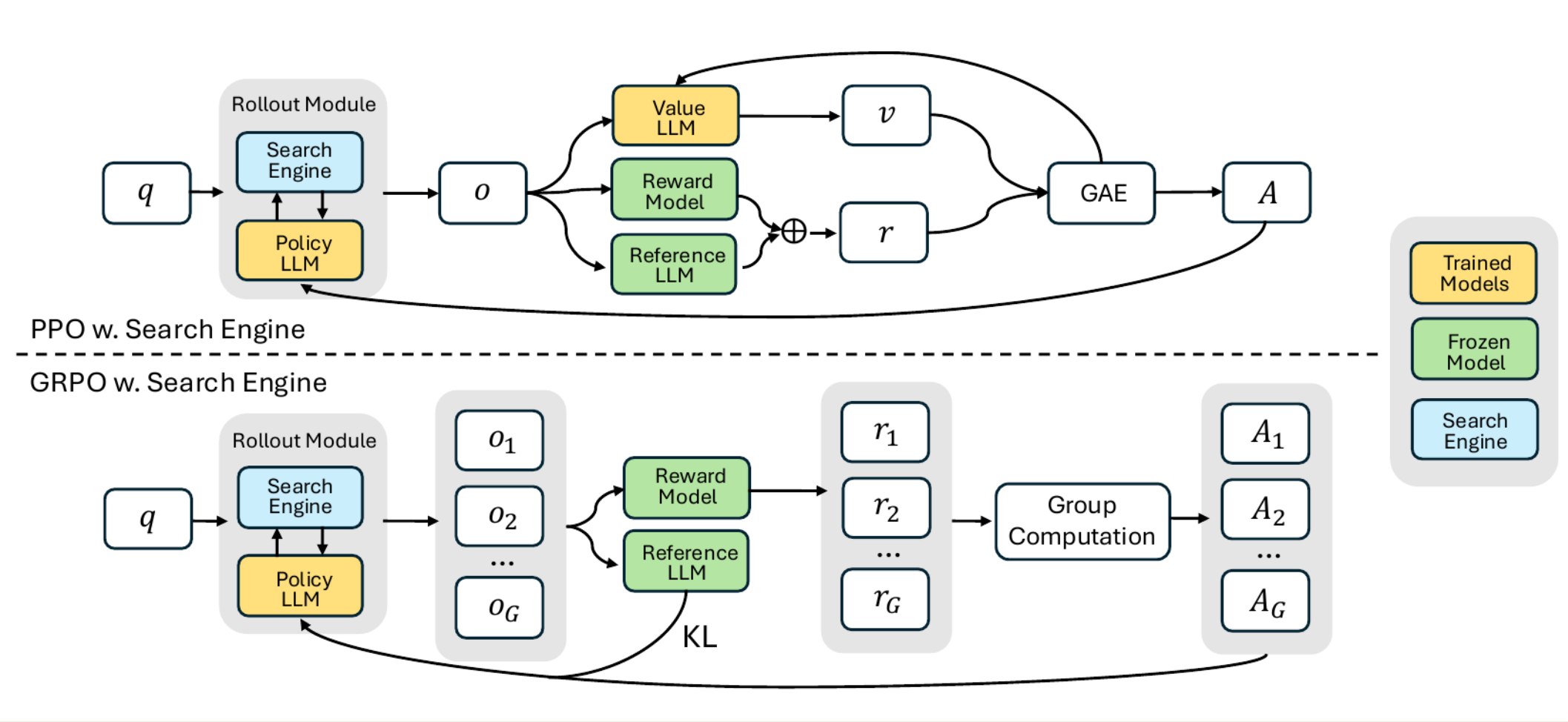
Question: Curious is a women's fragrance by a singer born in what city and state?
Ground Truth: McComb, Mississippi

R1:
<think> The singer is Beyoncé, who was born in Houston, Texas. </think>
<answer> Houston </answer>

SEARCH-R1:
<think> I need to find out which city and state a singer, by whom the women's fragrance "Curious" was created, was born in. </think>
<search> Curious fragrance information </search>
<information> Doc 1(Title: "Curious (fragrance)") Curious (fragrance) Curious is a women's fragrance by Britney Spears for Elizabeth Arden, and is the first perfume to be endorsed by Britney Spears, preceding ""Fantasy."" Spears reportedly earned $52 million from the endorsement. ""Curious"" was released in September 2004, and was very successful internationally. It was the number one fragrance of 2004 in department stores, and in 2005, ""Curious"" was honored by the Fragrance Foundation as Best Women's Fragrance. The fragrance is a white floral scent. Its notes are Louisiana magnolia, golden Anjou pear, lotus flower, tuberose, star jasmine, pink cyclamen, vanilla-infused musk, sandalwood and blonde woods. Elizabeth Arden ... </information>
<think> I need to find out which city and state Britney Spears, the singer behind "Curious" fragrance, was born in. </think>
<search> Britney Spears birthplace </search>
<information> Doc 1(Title: "Britney Spears") the best-selling female albums artist of the 2000s. In 2004, she launched a perfume brand with Elizabeth Arden, Inc., from which sales exceeded US$1.5 billion, . Spears was born in McComb, Mississippi, the second child of Lynne Irene Bridges and James Parnell Spears. Her maternal grandmother, Lillian Portell, was English (born in London), and one of Spears's maternal great-great-grandfathers was Maltese. Her siblings are Bryan James and Jamie Lynn. Britney was born in the Bible Belt, where socially conservative evangelical Protestantism is a particularly strong religious influence. Spears was baptized into the Southern Baptist Convention, but later in life studied </information>
<think> Since Britney Spears was born in McComb, Mississippi, I need to find out which city and state that is. </think>
<search> McComb, Mississippi location </search>
<information> Doc 1(Title: "McComb, Mississippi") McComb, Mississippi McComb is a city in Pike County, Mississippi, United States, approximately south of Jackson. As of the 2010 census, the city had a total population of 12,790. It is the principal city of the McComb, Mississippi Micropolitan Statistical Area. McComb was founded in 1872 after Henry Simpson McComb of the New Orleans, Jackson and Great Northern Railroad, a predecessor of the Illinois Central Railroad (now part of the Canadian National Railway), decided to move the railroad's maintenance shops away from New Orleans, Louisiana, to avoid the attractions of that city's saloons. The railroad purchased land in Pike County, </information>
<think> I have found that Britney Spears was born in McComb, Mississippi. Since the question asks for the city and state, I just need to confirm the state, which I already know from the previous search. </think>
<answer> McComb, Mississippi </answer>

Comparison of DeepSeek-R1 and SEARCH-R1
The model is trained to use tools

Search-R1: the complete training framework



q: question; o: attempt; r: reward; v: expected reward; A: advantage.

Jin et al. (2025), Figure 1: PPO above, GRPO below.

Does searching more help without training?

Same Qwen2.5-7B model and 2018 Wikipedia corpus; average over seven datasets.
IRCoT and Search-o1 prompt the model to search repeatedly; they do not train it.

Method	How it searches	Average exact match
RAG	Once, before answering	30.4%
IRCoT	Repeatedly, by **prompting**	23.9%
Search-o1	Repeatedly, by **prompting**	20.6%
Search-R1-instruct	Repeatedly, trained with rewards	38.4%

With this 7B model, **prompting it to search repeatedly** scored below searching once.
Training is **what made repeated search pay off**. More searching is not automatically better.

Selected Qwen2.5-7B values from Search-R1 v1, Table 2 (Jin et al., 2025).

05

Research on the open web

Can the agent still investigate when pages change or tools fail?

The assistant must now read live pages, beyond saved passages. Search snippets may omit evidence; pages may fail to load.

Three risks of reading the open web

Prompt injection

- A page can hide text written as instructions to the model.
- Treat page text as evidence to read, never as a command to follow.

Source credibility

- A high search ranking does not make a source reliable.
- Prefer primary sources, such as the original study, and note who published each.

Contamination

- Benchmark questions and answers can appear online.
- A web agent may find the answer key, so a high score may not reflect research skill.

Indirect prompt injection: Greshake et al. (2023).

DeepResearcher: training on the live web

During training

1. The research policy, the model being trained, sends **real web searches** and opens pages.
2. A separate **browsing agent** reads each selected page in segments and returns the relevant text.
3. That text joins the history the policy uses to choose its next action.
4. Completed attempts earn **rewards**, which train the research policy.

Before training

- The pipeline drops low-quality questions and ones the base model answers *without search*.

Live tool interaction, browsing roles and filtering: Zheng et al. (2025), §§3–4.

Why DeepResearcher matters

Real research is messy

- Real web pages can be cluttered, missing, or in conflict with each other.
- A clean Wikipedia snapshot, like the one Search-R1 used, tests none of that.

Training on the live web produced useful habits

- **Planning** before the first search.
- **Cross-checking** an answer against a second source.
- **Revising** the search after a poor result.
- **Saying so** when it cannot find a definite answer.

Behaviors observed in training: Zheng et al. (2025), abstract and Figure 2.

DeepResearcher: a trained model at work

Planning

Question: *What is the name of the famous bridge in the birthplace of Gloria in D Major's composer?*
Ground Truth: Rialto Bridge

Response Trajectory from DeepResearcher:

<think>
To answer this question, I need to figure out who Gloria in D Major's composer was and their birthplace. Then I can find the famous bridge in that city.

Step 1: Identify the composer of Gloria in D Major.
Step 2: Find the place of birth of that composer.
Step 3: Look for a famous bridge in that city.
</think>

Make a plan first

<search>
composer of Gloria in D Major
birthplace of the composer of Gloria in D Major
</search>
<tool_response>
{
'title': 'Gloria (Vivaldi) - Wikipedia',
'snippets': ' Antonio Vivaldi wrote at least three Gloria compositions, settings of the hymn Gloria in excelsis Deo, with words probably dating back to the 4th century.',
'url':'https://en.wikipedia.org/wiki/Gloria_(Vivaldi)'
}
{
'title': 'Antonio Vivaldi: Gloria in D Major, RV 589 - Boston Baroque',
'snippets': ' The Gloria, part of that Turin collection, received its twentieth-century premiere in 1930 and has remained the most popular of all Vivaldi's vocal works ever ...',
'url':'https://baroque.boston/vivaldi-gloria'
}
...(more results are omitted to save space)
</tool_response>

Step 1

<think>
From the search results, **I can see that Antonio Vivaldi is the composer of Gloria in D Major** and **he was born in Venice**. Now I need to find the famous bridge in Venice.
</think>

Step 2

<search>
famous bridge in Venice
</search>
<tool_response>
{
'title': 'THE 15 BEST Venice Bridges to Visit (2025) - Tripadvisor',
'snippets': 'Ponte della Paglia is a bridge that bridges the Rio di Palazzo canal on the border of the Sestiere San Marco and Cast',
'url':'https://www.tripadvisor.com/Attractions-g187870-Activities-c47-t5-Venice_Veneto.html'
}
{
'title': 'Rialto Bridge - Wikipedia',
'snippets': 'The Rialto Bridge is the oldest of the four bridges spanning the Grand Canal in Venice, Italy. Connecting the sestieri (districts) of San Marco and San Polo ...',
'url':'https://en.wikipedia.org/wiki/Rialto_Bridge'
}
...(more results are omitted to save space)
</tool_response>
<think>
The Rialto Bridge is the oldest and most famous bridge in Venice. I can now answer the question.
</think>

Step 3

<answer>
Rialto Bridge
</answer>

Tool calls are text.
The model writes a search; code runs it and returns results.

Example 1: planning. It writes a plan, adjusts it (Step 1, 2 are combined), and merges steps.

Figure 2: planning and cross-validation, hand-picked examples; Zheng et al. (2025).

DeepResearcher: a trained model at work

Cross Validation

Question: *Which filmmaker is considered a pioneer, Herschell Gordon Lewis or Leah Meyerhoff?*
Ground Truth: *Herschell Gordon Lewis was a pioneer*

Response Trajectory from DeepResearcher:

<think>
To answer this question, I need to investigate the contributions of both Herschell Gordon Lewis and Leah Meyerhoff to filmmaking. Specifically, I need to understand if either of them is considered a pioneer. First, I will perform a web search to find out more about the filmmakers and what they are known for.
</think>
<search>
Herschell Gordon Lewis filmmaker pioneer
Leah Meyerhoff filmmaker pioneer
</search>
<tool_response>
{
'title': 'Herschell Gordon Lewis - Wikipedia',
'snippets': 'Herschell Gordon Lewis (June 15, 1926 September 26, 2016) was an American filmmaker, best known for creating the 'splatter' subgenre of horror films.',
'url':'https://en.wikipedia.org/wiki/Herschell_Gordon_Lewis'
}
...(more results are omitted to save space)
</tool_response>
<think>
The web search results show that Herschell Gordon Lewis is frequently referred to as a pioneer in the genre of horror films, often being called the "Godfather of Gore" and "Pioneer in the arena of gore or splatter flicks." On the other hand, Leah Meyerhoff is described as an award-winning independent filmmaker, particularly known for her narrative feature film "I Believe In Unicorns". There is no mention of her being a pioneer in the filmmaking industry.

Based on the information retrieved, it seems that Herschell Gordon Lewis is the filmmaker considered a pioneer.

The next step is to narrow down the exact reason why he is considered a pioneer. I will conduct another web search with more specific keywords to **pinpoint the characteristics of his work that set him apart as a pioneer**.
</think>
<search>
Herschell Gordon Lewis why considered a pioneer
Herschell Gordon Lewis pioneer features
</search>
<tool_response>
{
'title': 'Herschell Gordon Lewis, a Pioneer of Gore Cinema, Dies at 90',
'snippets': 'Mr. Lewis set the bar for low-budget horror in movies like "Blood Feast" that went on to inspire filmmakers like Wes Craven and Quentin ...',
'url':'https://www.nytimes.com/2016/09/28/movies/herschell-gordon-lewis-a-pioneer-of-gore-cinema-dies-at-90.html'
}
...(more results are omitted to save space)
</tool_response>
<think>
From the refined search, it is clear that Herschell Gordon Lewis is considered a pioneer due to his pioneering the "splatter" subgenre of horror films. His films introduced the extreme visuals and violent content that set them apart, even though they were often considered an "insult to the art of filmmaking". Therefore, Herschell Gordon Lewis is the filmmaker considered a pioneer.
</think>
<answer>
Herschell Gordon Lewis
</answer>

Validate result from difference sources

Tool calls are text. The model writes a search; code runs it and returns results.

Example 2: checking. It finds the answer early, then searches again to confirm it.

Figure 2: planning and cross-validation, hand-picked examples; Zheng et al. (2025).

The metric can change the apparent winner

Natural Questions supplied training data. The table tests separate development questions.
F1 scores word overlap. GPT-4o-mini judges answer meaning. Higher scores are better.

System	F1 (%)	Model-judged correctness (%)
Search-r1-base, local retrieval	**45.4**	60.0
DeepResearcher, web search	39.6	**61.9**

DeepResearcher scores higher under the model judge, but lower on word overlap.
Before trusting either number, read the answers and the cited evidence.
The judge can make mistakes, and the two systems search different sources.
DeepResearcher is searching the open Web versus local retrieval

Selected Natural Questions values from Table 1.

06

Producing new research results

What changes when the agent must produce a new result?

A report summarizes existing work. A mathematics research agent also proposes arguments that may not appear in its sources.
We now turn to how an agent checks those arguments.

Evaluating a mathematical result

- **Correctness** asks whether the argument's steps follow from its assumptions.
  - A fluent explanation can still contain a missing or invalid step.
- **Novelty** asks what the result adds to existing work.
  - A correct proof of a known theorem is correct, but it is not new mathematics.
- Human **contribution** asks who supplied the ideas, repaired the argument, and checked it.
  - The finished text alone does not show who did the thinking.

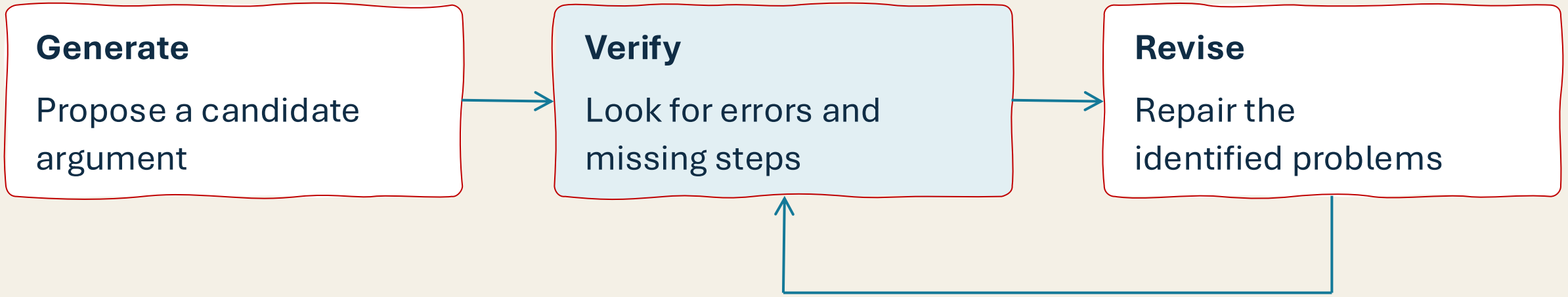
Research includes ideas, discussion, and checking.



We need **separate evidence** for each claim about a research agent's achievement.

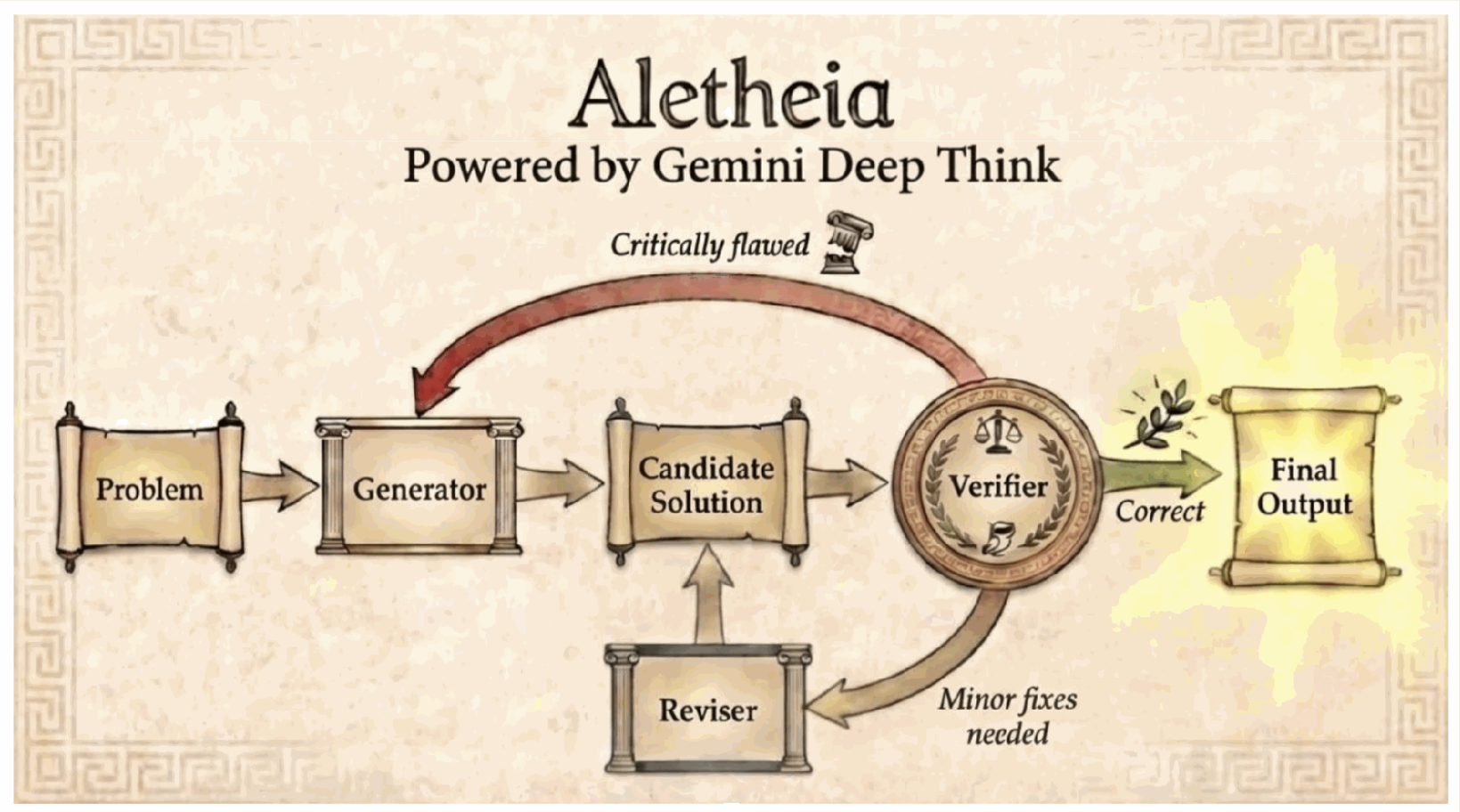
Generating, verifying, and revising

A candidate proof is a proposed argument that **still needs checking**.
A model can inspect that argument separately and request a revision.



The reviser sends its repaired argument back for another verification.
The system stops after **approval** or when its **attempt limit** is reached.

Aletheia's original agent loop



How the loop works

- The **generator** writes a candidate proof.
- The **verifier** checks it. A correct proof is final.
- **Minor errors:** the reviser repairs them.
- **Serious flaws:** the generator starts again.

Caution

- The verifier is also a model, so an approved proof can still be wrong. Experts must still check it.

Figure 1: Aletheia orchestration, Feng et al. (2026).

Generate-verify-revise is not new

Early AI called this pattern “generate and test.” Self-Refine (2023) is a recent example.

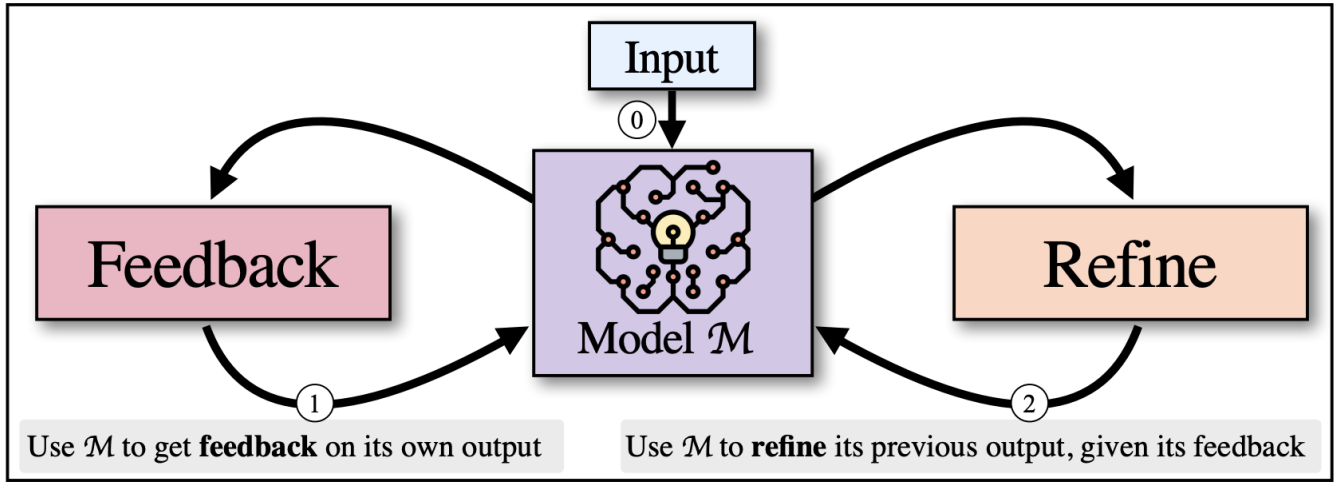


Figure 1: Given an input (⓪), SELF-REFINE starts by generating an output and passing it back to the same model $\mathcal{M}$ to get feedback (①). The feedback is passed back to $\mathcal{M}$, which refines the previously generated output (②). Steps (①) and (②) iterate until a stopping condition is met. SELF-REFINE is instantiated with a language model such as GPT-3.5 and does not involve human assistance.

Like Aletheia: one model writes an output, critiques it, and rewrites it, with no human help.

Figure 1: Madaan et al. (2023), Self-Refine: Iterative Refinement with Self-Feedback.

Self-Refine in practice: two examples

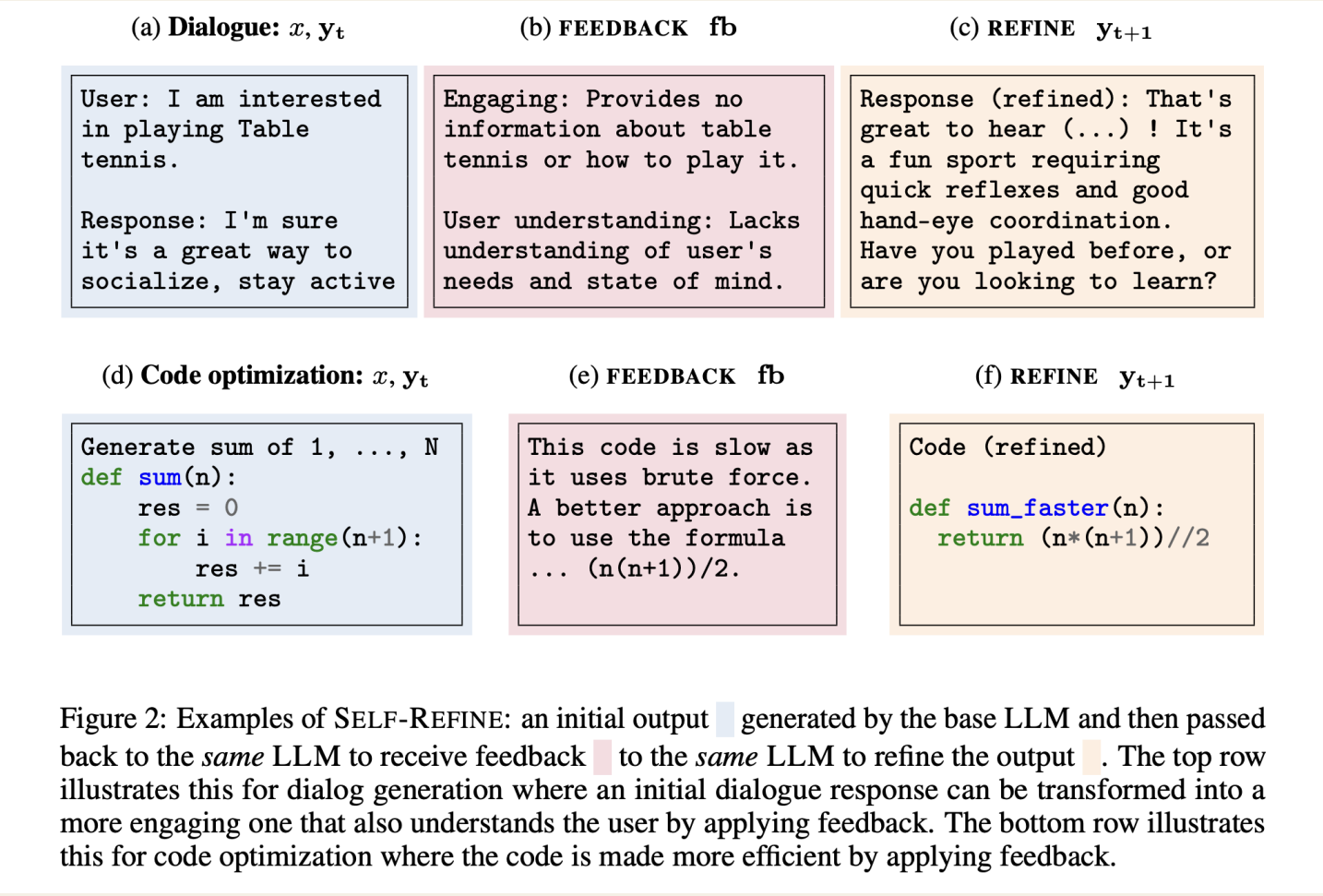


Figure 2: Examples of SELF-REFINE: an initial output generated by the base LLM and then passed back to the *same* LLM to receive feedback to the *same* LLM to refine the output . The top row illustrates this for dialog generation where an initial dialogue response can be transformed into a more engaging one that also understands the user by applying feedback. The bottom row illustrates this for code optimization where the code is made more efficient by applying feedback.

Two examples

- **Top row:** a chat reply.
- **Bottom row:** slow code.

Two steps, one model

- **Feedback** names the problem: the reply ignores table tennis; the code adds every number one by one.
- **Refine** rewrites the output to fix it.
- The **same model** does both steps, with no human help.

Figure 2: Madaan et al. (2023), Self-Refine examples.

Aletheia: A hallucinated paper

Sample Interaction

Prompt: Prove or disprove: the pretzel knot $P(-3, 5, 13)$ has infinite order in the smooth concordance group.

Model output:

...

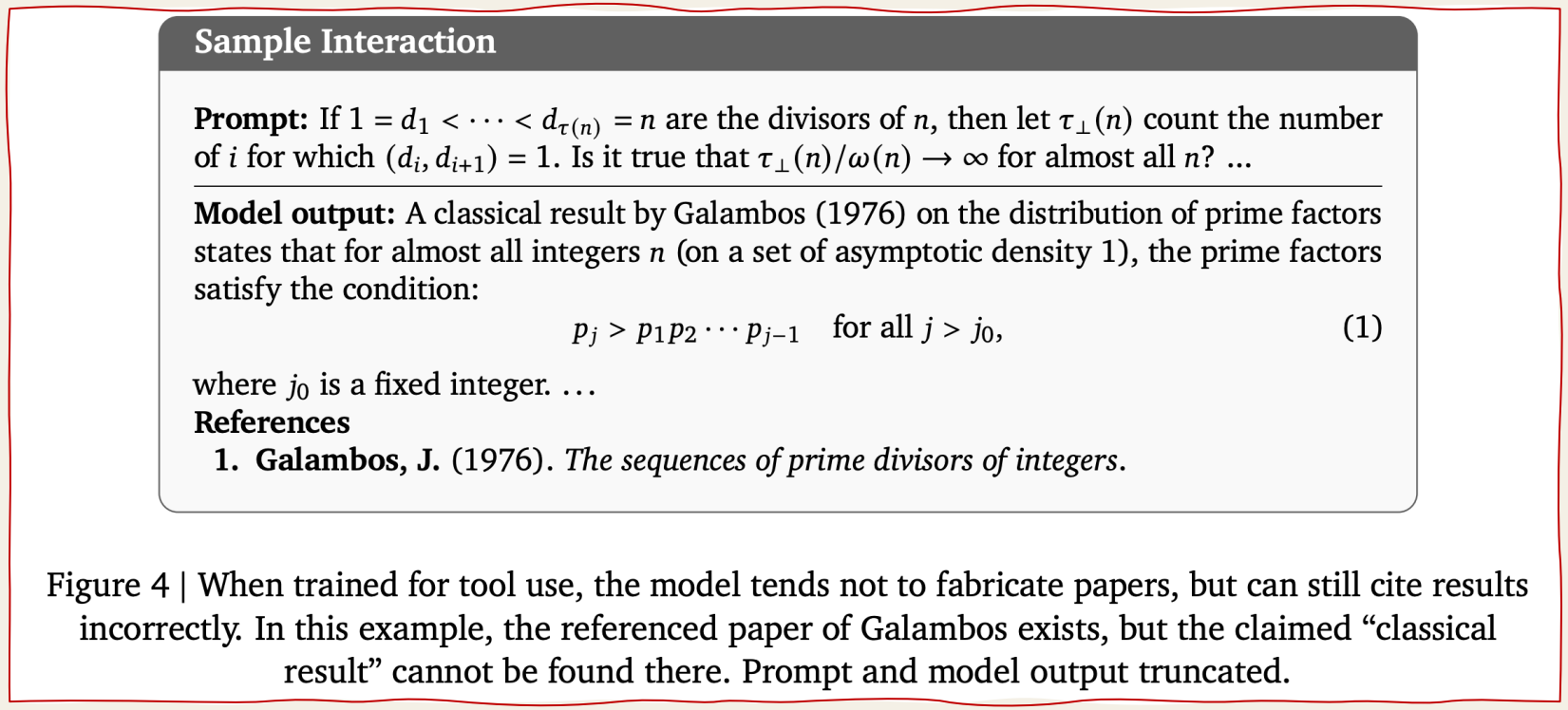
Proof. This is derived from Theorem 3.1 in [C. Livingston and S. Naik, "Ozsváth-Szabó and Rasmussen invariants of some pretzel knots", *Algebraic & Geometric Topology*, 13(2) (2013), 1115-1124].

...

Figure 3 | A hallucinated paper from the (truncated) output of a model without internet search capability. The red text refers to a completely fabricated paper.

Figure 3: a fabricated citation from a model without web search; Feng et al. (2026).

Aletheia: An existing reference can still fail to support a claim



Sample Interaction

Prompt: If $1 = d_1 < \cdots < d_{\tau(n)} = n$ are the divisors of n, then let $\tau_\perp(n)$ count the number of i for which $(d_i, d_{i+1}) = 1$. Is it true that $\tau_\perp(n)/\omega(n) \to \infty$ for almost all n? ...

Model output: A classical result by Galambos (1976) on the distribution of prime factors states that for almost all integers n (on a set of asymptotic density 1), the prime factors satisfy the condition:

$$p_j > p_1 p_2 \cdots p_{j-1} \quad \text{for all } j > j_0, \tag{1}$$

where j_0 is a fixed integer. ...

References

1. **Galambos, J.** (1976). *The sequences of prime divisors of integers*.

Figure 4 | When trained for tool use, the model tends not to fabricate papers, but can still cite results incorrectly. In this example, the referenced paper of Galambos exists, but the claimed "classical result" cannot be found there. Prompt and model output truncated.

Citation failure

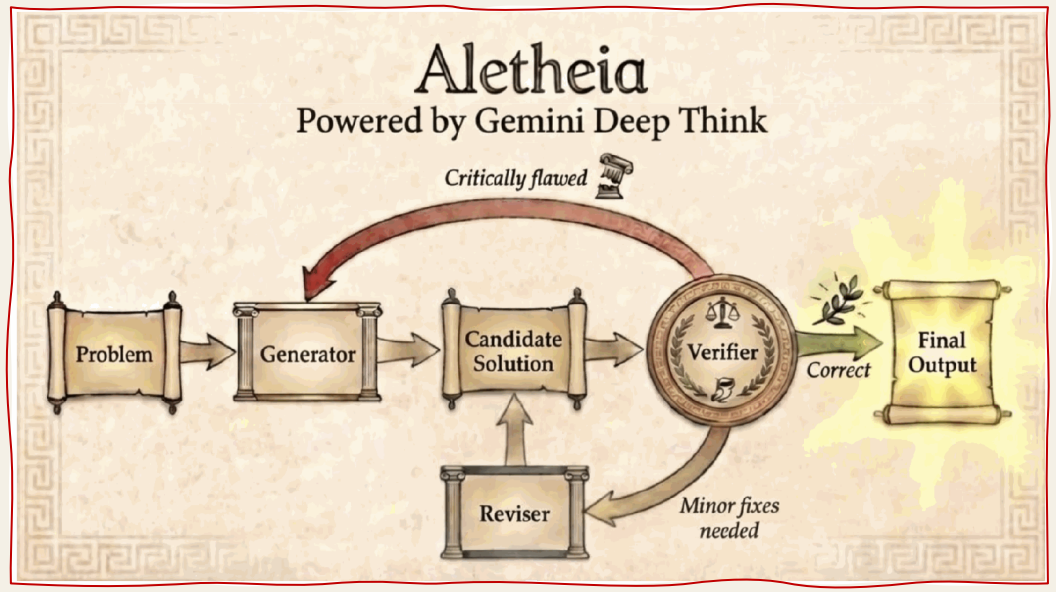
- The model attributes a result about prime factors to **Galambos (1976)**.
- The **paper exists**, but the authors **could not find** that result in it.

With web search, Aletheia stopped inventing papers, but it still cited real papers for results they do not contain.

Figure 4: a real source, an unsupported attribution.

What Aletheia contributes

- **Research results:** with a verifier and web search, the model produced checkable research mathematics, including solutions to open Erdős problems.
- **A separate verifier helps:** with the same base model, Aletheia beat Deep Think and could decline to answer instead of guessing.
- **Honest reporting:** the authors rate each result by its novelty and by how much the AI did. The next slides show these ratings.
- **Not new:** the generate–verify–revise loop itself. Similar designs existed before.



Feng et al. (2026), §1–2 and §5.

Aletheia: what does the score count?

Aletheia can skip a problem it cannot solve, so there are two ways to score it:

- **Overall score:** points earned across **all** problems. A skipped problem earns 0.
- **Conditional accuracy:** points earned on **only the problems it answered**.

IMO-ProofBench Advanced: 30 olympiad-level proof problems. Aletheia answered 29.

Overall score

95.1%

Counts all 30 problems

Conditional accuracy

98.3%

Counts only the 29 it answered

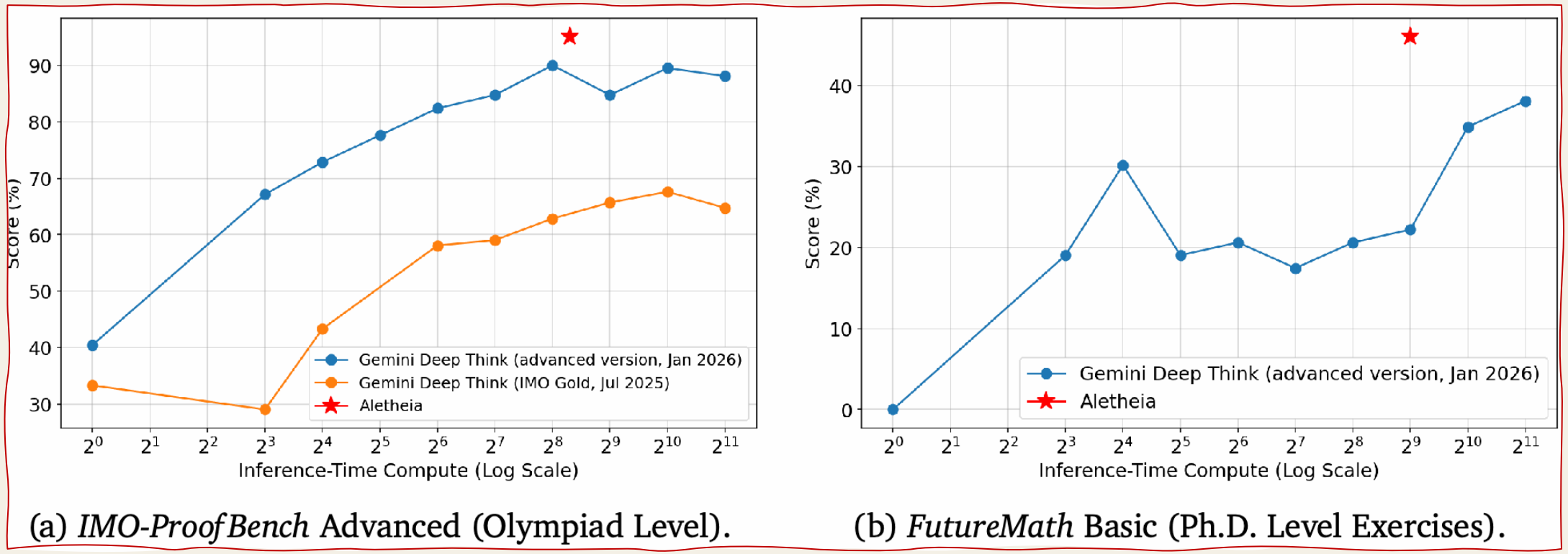
Why both numbers matter: FutureMath Basic (doctoral-level problems)

- Over 82% correct on the problems it answered, but it answered fewer than 60% of them.
- Count the skipped problems as zero, and the overall score is roughly half ($82\% \times 60\% \approx 49\%$).

Feng et al. (2026), §2.2: overall score and conditional accuracy.

Aletheia: score and solving effort

Scores reflect human grading across all problems. Higher scores are better.
Each horizontal tick **doubles** the computing effort. Red stars mark Aletheia.



(a) *IMO-ProofBench* Advanced (Olympiad Level). (b) *FutureMath* Basic (Ph.D. Level Exercises).

Aletheia beats the blue curves: the same model, Deep Think, run without Aletheia's loop.
Compute varies across attempts, so the experiments do not hold solving effort *exactly equal*.

Figure 2; Feng et al. (2026), §§2.1–2.2.

07

Choosing and testing a research agent

What would justify using the more complex agent?

A report, an answer, and a candidate proof need different checks. Compare the evidence gained with the complexity and cost added.

Choosing a research-agent design

Match the design to the problem:

- **Your first search often misses what you need** → search, read, search again (ReAct)
- **You need a broad report on a topic** → research first, then outline (STORM)
- **The question is about published research** → search scientific papers (PaperQA2)
- **You have practice questions with answers** → train search with rewards (Search-R1)
- **Answers live on real, changing web pages** → train on the live web (DeepResearcher)
- **The output is a mathematical proof** → generate, check, revise (Aletheia)

Start from the failure you see, then pick the design that fixes it.

How to read a research-agent result

Before trusting a reported gain, ask three questions:

What was counted?

Aletheia scored 98.3% on the 29 problems it answered, but 95.1% on all 30.

What was held out?

DeepResearcher was never trained on Bamboogle questions, so its score there shows real generalization.

What else differed?

DeepResearcher searched the live web; Search-R1 searched a 2018 Wikipedia copy. The gap mixes training with sources.

Testing your own agent: does searching again help?

A fair test

- Run both versions on the same questions, none of them used for tuning.
- Same model, sources, and budget. The only difference: one may search again.

Record for each run

- Is the answer correct, and do the cited passages support it?
- Did it admit missing evidence instead of guessing?
- What did it cost in tool calls and time?

Decision: searching again is worth it only if it improves the answer more than it costs.

What a good research agent does

1. **Investigates until the evidence is enough**
   Searches, reads, follows citations, and searches again.
2. **Claims no more than the evidence shows**
   “No significant change” in one study is not “no effect.”
3. **Backs every claim with a source that supports it**
   A citation counts only if the passage says what is claimed.
4. **Names the gap when the evidence runs out**
   Then searches for what is missing, if the budget allows.
5. **Stays within a budget**
   Every extra search costs time and money; investigation can’t run forever.

A good answer is not just correct.

It is supported, and honest about what is missing.

References: prompted research systems

ReAct

Yao et al. (2023). Synergizing Reasoning and Acting in Language Models.

STORM

Shao et al. (2024). Assisting in Writing Wikipedia-like Articles From Scratch with Large Language Models.

PaperQA2

Skarlinski et al. (2024). Language agents achieve superhuman synthesis of scientific knowledge.

Self-Refine

Madaan et al. (2023). Self-Refine: Iterative Refinement with Self-Feedback. NeurIPS 2023.

References: training and autonomous research

Search-R1

Jin et al. (2025). Training LLMs to Reason and Leverage Search Engines with Reinforcement Learning.

DeepResearcher

Zheng et al. (2025). Scaling Deep Research via Reinforcement Learning in Real-world Environments.

Aletheia

Feng et al. (2026). Towards Autonomous Mathematics Research.

References: adaptive search and deployment

FLARE

Jiang et al. (2023). Active Retrieval Augmented Generation. EMNLP 2023.

Self-RAG

Asai et al. (2024). Self-RAG: Learning to Retrieve, Generate, and Critique through Self-Reflection. ICLR 2024.

Adaptive-RAG

Jeong et al. (2024). Adaptive-RAG: Learning to Adapt Retrieval-Augmented Large Language Models through Question Complexity. NAACL 2024.

Multi-agent research

Hadfield et al. (2025). How we built our multi-agent research system.

References: citation quality and web risks

ALCE

Gao et al. (2023). Enabling Large Language Models to Generate Text with Citations. EMNLP 2023.

Indirect prompt injection

Greshake et al. (2023). Not what you've signed up for: Compromising Real-World LLM-Integrated Applications with Indirect Prompt Injection. arXiv:2302.12173.